AI BASED INVOICE ANALYZER USING CRM AUTOMATION

A PROJECT REPORT

Submitted by

MUKESH N	**111922AI01060**
RAJ KUMAR V	**111922AI01076**
SHAMGANESH S	**111922AI01091**
SOMESHWUR R	**111922AI01096**

in partial fulfillment for the award of the degree

of

B. TECH

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

S. A. ENGINEERING COLLEGE, CHENNAI - 600077
(Autonomous - Institute Level Research Centre, Affiliated to Anna University)

ANNA UNIVERSITY : CHENNAI 600 025

NOVEMBER 2025

BONAFIDE CERTIFICATE

Certified that this project report **"AI BASED INVOICE ANALYZER USING CRM AUTOMATION"** is the bonafide work of "**MUKESH N (111922AI01060), RAJ KUMAR V (111922AI01076), SHAMGANESH S (111922AI01091)** and **SOMESHWUR R (111922AI01096)**" who carried out the project work under my supervision.

SIGNATURE OF HoD
HEAD OF THE DEPARTMENT
Dr. D HEMANAND, M.E., PhD.,
Professor & Head,
DEPARTMENT OF AI & DS,
S.A.ENGINEERING COLLEGE,
Avadi – Poonamallee Main Road,
Thiruverkadu Post,
Chennai – 600 077

SIGNATURE OF SUPERVISOR
SUPERVISOR
Mrs. B. ARDLY MELBA REENA, M.,E., (PhD).,
Assistant Professor,
DEPARTMENT OF AI & DS,
S.A.ENGINEERING COLLEGE,
Avadi – Poonamallee Main Road,
Thiruverkadu Post,
Chennai – 600 077

Submitted to Project Viva-Voice Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We owe a great many thanks to a great many people who helped and support us during the completion of our project.

We take this opportunity to express our profound gratitude and deep regards to our-Founder Chairman **(Late)** Shri**. D.SUDHARSSANAM, M.L.A,** our Chairman Shri**. D. DURAISWAMY**, our humble Secretary Shri**. D. DASARATHAN** our correspondent Shri. **S. AMARNAATH**, and our Director Shri**. D. SABARINATH**, for their exemplary guidance, monitoring and constant encouragement throughout the course of this thesis. The blessing, help and guidance given by them time to time shall carry us a long way in the journey of life on which we are about to embark.

We also take this opportunity to sincerely thank our Advisor **Prof. Dr. S. SALIVAHANAN, M.E., Ph.D.,** for his guidance. Also we express a deep sense of gratitude to thank our beloved Principal **Dr. S. RAMACHANDRAN, M.E., Ph.D.,** for extending his support. Also we sincerely thank **Dr. D. HEMANAND, M.E., Ph.D.,** Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement in our project work which helped us in completing this task through various stages.

We are also obliged to convey our special and sincere gratitude to our project coordinators, **Mrs. B. ARDLY MELBA REENA, M.E., (PhD).,** Assistant Professor, AI & DS Department, **Mr. G. MUTHUKUMAR, M.E., (PhD).,** Assistant Professor, AI & DS Department, and our internal guide, **Mrs. B. ARDLY MELBA REENA, M.E., (PhD).,** AI & DS Department, for their encouragement and concern during the review session, along with their valuable advice, knowledge, and expert guidance in our project.

Lastly, we thank almighty, our parents, all the teaching and non-teaching staff for their support and guidance throughout our project and for their constant encouragement without which this project would have not been possible.

ABSTRACT

The proposed project introduces an AI-powered web application designed to automate invoice processing and seamlessly integrate business operations with CRM systems. The system employs Optical Character Recognition (OCR) to extract key details from invoices, validates the data for accuracy, and intelligently matches vendor or customer information with CRM records using fuzzy logic. It further enhances business efficiency by generating AI-based upsell recommendations from historical purchase data. All processes are managed through a unified, user-friendly dashboard, eliminating manual work, minimizing errors, and ensuring faster decision-making. This end-to-end solution bridges the gap between document processing and business automation, offering improved accuracy, operational efficiency, and revenue opportunities for modern enterprises. By combining AI with enterprise workflows, the project demonstrates how intelligent automation can transform traditional financial operations into a smart, scalable, and future-ready system.

TABLE OF CONTENTS

S. NO	TITLE	PAGE NO
	ABSTRACT	**IV**
	LIST OF CONTENTS	**V**
	LIST OF FIGURES	**VII**
	LIST OF TABLES	**VIII**
	LIST OF ABBREVIATION	**IX**
1	**INTRODUCTION**	**1**
	1.1. Overview	1
	1.2. Problem Statement	1
	1.3. Aim & Objective	2
	1.4. Scope	3
2	**LITERATURE SURVEY**	**4**
3	**SYSTEM ANALYSIS**	**9**
	3.1. Existing System	9
	3.2. Proposed System	11
	3.3 Comparison Table	13
4	**SOFTWARE CONFIGURATION**	**14**
	4.1. Hardware Requirements	14
	4.2. Software Requirements	15
	4.3. Software Description	15
5	**SYSTEM DESIGN**	**22**
	5.1. System Architecture	22
	5.2. ER Diagram	23
	5.3. OCR Extraction Template	24
	5.4. Database Schema	25
6	**SYSTEM IMPLEMENTATION**	**27**
	6.1. System Implementation	27
	6.2. Modules Description	29
7	**SOFTWARE TESTING**	**35**

7.1 Accuracy, Precision, Recall, and F1-Score 35

7.2. Performance Evaluation Table 36

7.3. Comparative Analysis with Manual Processing 37

8 CONCLUSION AND FUTURE ENHANCEMENT 39

8.1. Conclusion 39

8.2. Future Enhancement 39

APPENDIX 1: SNAPSHOTS 41

APPENDIX 2: SOURCE CODE 44

REFERENCES 59

LIST OF FIGURES

FIGURE NO.	FIGURES	PAGE NO
5.1	System Architecture	22
5.2	ER Diagram	23
5.3	OCR Extraction Template	24
6.1	Invoice Upload Page	33
6.2	Invoice Extracted and Display Page	34
7.3.1	Compariative Analysis	38
7.3.2	Performance Evaluation	38
Appendix1.1	Upload Invoice	41
Appendix1.2	Invoice Dashboard	41
Appendix1.3	Customer Dashboard	42
Appendix1.4	Vendor Dashboard	42
Appendix1.5	Analysis Dashboard	43
Appendix1.6	Upsell Suggestion Dashboard	43

LIST OF TABLES

TABLE NO.	TABLES	PAGE NO
3.3	Comparison Table	13
5.4.1	Invoice Table	25
5.4.2	Vendor Table	25
5.4.3	List Item Table	26
5.4.4	CRM Mapping Table	26
5.4.5	Notification Table	26
7.2	Performance Evaluation Table	36
7.3	Comparative Analysis with Manual Processing	37

LIST OF ABBREVIATION

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
OCR	Optical Character Recognition
CRM	Customer Relationship Management
DFD	Data Flow Diagram
ER	Entity Relationship
API	Application Programming Interface
DBMS	Database Management System
SQL	Structured Query Language
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
PDF	Portable Document Format
ANN	Artificial Neural Network
CNN	Convolutional Neural Network

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Invoice processing is a fundamental financial operation in businesses, involving the receipt, verification, and recording of invoices issued by vendors or suppliers. Traditionally, this process has been manual, requiring finance or accounting teams to extract data from paper or digital invoices. Manual processing is time consuming, prone to human errors, and often results in delayed payments or discrepancies. With the growth of digital business operations, companies have started adopting automation and AI technologies to streamline invoice handling. Modern invoice processing uses Optical Character Recognition (OCR) to convert invoice documents into machine readable formats and Artificial Intelligence (AI) and Natural Language Processing (NLP) to extract relevant data accurately. Integrating these systems with Customer Relationship Management (CRM) platforms ensures that invoice information is validated, recorded, and tracked efficiently, reducing operational costs and improving vendor relationships. In essence, AI-driven invoice processing transforms a traditionally repetitive and error-prone task into a fast, accurate, and automated workflow, enhancing overall financial management.

1.2 PROBLEM STATEMENT

Despite advances in digital technologies, many organizations still rely on manual or semi-automated invoice processing, which is time-consuming and prone to errors. **Challenges include:**

- High error rates due to manual data entry and human oversight.
- Multiple invoice formats (PDFs, scanned images, emails) making

standardization difficult.

- Inefficiency in handling large volumes of invoices, causing delays in approvals and payments.
- Integration issues with CRM systems, leading to data discrepancies and lack of real-time updates.
- Limited transparency, making it hard to track invoice status or detect duplicates.

The problem, therefore, is the need for an intelligent, automated system that can accurately extract invoice data, validate it against CRM records, and streamline the entire invoice processing workflow, reducing errors, saving time, and improving operational efficiency.

1.3 AIM AND OBJECTIVES

The aim of this project is to develop an AI-Based Invoice Analyzer Using with CRM Automation to streamline and enhance the invoice processing workflow. The system is designed to automatically capture, extract and validate invoice data from various formats and ensure seamless integration with CRM Platforms.

Objectives:

The OCR (Optical Character Recognition) and NLP (Natural Language Processing) techniques are used to extract structured data from unstructured invoice documents like scanned images, PDFs, and digital formats. It ensures accuracy by validating extracted invoice data with CRM records, such as vendor details and purchase orders. It integrates with the CRM platform for automatic updating of invoice details and cut down the average invoice processing time significantly compared with the traditional methods. To reduce the processing time

and human errors in invoice handling. To provide real-time updates, notifications, and reports through CRM integration. To design a scalable system that can process varied invoice formats (PDF, images, scans).

1.4 SCOPE

- The system will process invoices in multiple formats, including PDFs, scanned copies, and images.
- It will extract critical invoice fields such as vendor ID, invoice number, invoice date, payment terms, line items, taxes, and total amounts.
- The system will integrate with CRM platforms (e.g., Salesforce, Zoho, HubSpot, or custom CRMs) to automatically update invoice records.
- Automation will extend to workflows including approvals, duplicate detection, notifications, and data updates.
- Real-time dashboards and analytics will provide insights into invoice processing, pending approvals, and exception handling.
- The system will allow semi-automated intervention for complex cases or data mismatches requiring human review.

CHAPTER 2
LITERATURE SURVEY

Kumar Subham, et. al, 2025 [1] - Traditional CRM limited; AI enables real-time analysis / personalization. Retention as strategic priority. CRM in GTM/revenue ops Aligns functions, data source for optimization, personalization/automation needs. AI transforms CRM Technologies (NLP / ML / chatbots), workflows / decisions, segmentation / profiling, real-time analysis. Predictive analytics in retention Churn / LTV / upsell use cases, tools (regression / neural nets / trees), impact on loyalty. Automation Lead scoring, personalized comms, trigger journeys, workflows. Partner enablement PRM role, performance analysis, personalized support, case. Cases Salesforce (Einstein: 35% retention boost), Amazon (recommendations: 60% revenue). Roadmap Planning / steps, tools / platforms, challenges (data / adoption), scaling tips. Trends Generative / autonomous AI, copilots, vendor ecosystems. AI mandatory for CRM; ethical data / strategic alignment key. Examines ML/NLP for lead scoring and GTM strategies. Generative AI copilots for autonomous engagement. Overcomes data silos, enabling predictive interventions and partner ecosystems for revenue ops. The research shows how AI transforms traditional CRM into intelligent systems for better retention, GTM strategies, and revenue ops through automation and personalization. Case studies like Salesforce demonstrate improved lead scoring and preemptive engagement, addressing challenges like data quality and adoption, while forecasting trends in generative AI for scalable, autonomous customer management.

Aziz Amari, Mariem Makni, et. al, 2025 [2] - Large orgs process many invoices; authenticity via signature/stamp/fields. Challenges: Layout/quality variability. DL for text recognition/object detection via transfer learning. Materials/Methods 381 docs (240 machine-written), annotated (title / client / stamp / signature / date / total / etc.). Blur / jitter / padding / rotation. OCR (Azure / Tesseract), layout (LayoutLMv2 / v3 / LiLT), detection (RetinaNet / FCOS / Faster R-CNN / SSD). Results OCR: Azure selective/stable. Layout: LayoutLMv3 best F1 (0.56), but overfitting; LiLT faster. Detection: RetinaNet highest mAP (83.96@0.50). Pipeline: Azure + LayoutLMv3 + RetinaNet, Hyperparams, handwritten invoices. Presents a DL pipeline for validating scanned/handwritten invoices: Layout analysis (LayoutLMv3) + object detection (Faster R-CNN) on a novel annotated dataset (1,000+ real invoices). Detects signatures/stamps with 92% mAP, handling distortions via transfer learning. Automates reimbursement in large orgs, reducing manual review by 85% and errors. Positive impacts on personalization (mean 4.2/5), predictive accuracy (4.0), automation (3.8). Regression shows interdependencies. Discussion: aligns with literature on agility/readiness; fills empirical gap. Implications: practical adoption, theoretical validation, strategic advantage

Jayesh Jhurani, et. al, 2024 [3] - AI transforms CRM in ERP, enabling personalized, predictive interactions. Significance: Meets demand for differentiation. Studies show AI fosters customer-centricity, automates processes. Reviews works on AI-CRM adoption, readiness, performance, security. Gap: Empirical evidence on personalization/prediction/automation in India. Quantitative design, questionnaire to 250 professionals (April-June 2022), SPSS analysis (descriptive/inferential, regression). Demographics (60% CRM managers, varied industries/experience). Perceptions high. Regressions show significant

relationships ($p<0.001$). Aligns with literature, bridges gap. Implications: Practical (invest in AI), theoretical (empirical validation), strategic (competitive edge). AI enhances CRM in ERP, urging adoption for customer centricity. Positive impacts on personalization (mean 4.2/5), predictive accuracy (4.0), automation (3.8). Regression shows interdependencies. Discussion: aligns with literature on agility/readiness; fills empirical gap. Practical adoption, theoretical validation, strategic advantage.

Srinivasa Rao Gunturu, et. al., 2024 [4] - FinTech integration for automatic payments. ERP evolution (ERP I-III) incorporates cloud/AI/blockchain for efficiency. Features: profit tracking, ledger/cash flow management, payments. Integrations with gateways/portals enable seamless transactions; blockchain/Tripple-entry accounting adds security. Comparison: NetSuite vs. JDE/SAP/BaaN. Improves customer experience; needs more implementation studies. Explores ERP evolution (ERP I–III) with FinTech integrations like cloud/AI for seamless payments (e.g., Oracle Netsuite). Covers gateways, portals, and blockchain for fraud-proof triple-entry accounting. Highlights real-time CRM-ERP links for cash flow prediction. Implication: Enables global scalability, slashing errors by 50% and enabling predictive analytics for vendor payments, vital for e-commerce growth. The introduction discusses online transactions making manual transfers costly and error-prone. ERP payment systems with portals/gateways enable seamless automation. ERP consolidates data from sources like HR, manufacturing, and finance. FinTech provides ERP with payment integration. It covers ERP history from 1970, coined in 1990 by Gartner. Evolution: ERP I (integration), ERP II (extended), ERP III (customer/supplier integration for innovation) on modern ERP payments: Cloud ERP reduces costs (e.g., Oracle ERP, Syspro). AI (iERP) processes unstructured data, simplifies

workflows. Real-time support via ERP-CRM integration for decisions. The details ERP payment systems: profit tracking (health evaluation), ledger management (connects modules), cash flow control (accounts payable), payments tracking (gateways for views/reminders). It is on Front/back-end for seamless processing. ERP III enables customer/vendor access. Blockchain for triple-entry accounting (secure, fraud-proof). Autopay/auto-receive modules automate based on terms. The examines Netsuite: Cloud ERP with 19,000+ clients. Integrates payables, approvals, reports. Modules: procure-to-pay, purchase order, item receipt, bill payment, inventory management. It compares SAP, BaaN, Oracle (JDE), Netsuite: Netsuite excels in reporting, support, user-friendliness. Integration streamlines finance. More research on implementations/cases needed.

Adem Akdogan, Murat Kurt, et. al., 2024 [5] - The paper introduces ExTTNet, a multi-layer artificial neural network (ANN) for autonomously extracting product table texts from invoice images to streamline accounting processes. Using Tesseract OCR on 8794 German invoices, it extracts texts and enhances features (e.g., alignments, patterns, margins) via engineering, labeling table elements (1) vs. non-table (0). The 8-layer model (ReLU hidden, sigmoid output) trains in 162 minutes on an Nvidia RTX 3090, achieving an F1 score of 0.92 (precision/recall ~0.92-0.93). Contributions include the novel model, feature utilization for accuracy, and validation showing time savings; future work involves image integration and optimizations for better handling of noise/angles. This advances rule-based/OCR methods by focusing on text-level classification over block detection, reducing manual entry needs in high-volume operations. Extracts tables via OCR + features + ANN. Tesseract on 8794 invoices; labeling 1/0; training 162 min; F1 0.92. DL for document ops; ExTTNet for invoices. Related - Text-based (rule/DL/ML); Image-based (object/rule).

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In most organizations, invoice processing is handled through manual or semi-automated methods. The current system is primarily dependent on human intervention, which makes the process time-consuming, error-prone, and inefficient.

i. **Manual Data Entry:**

Invoices, whether received in physical paper format or as digital PDFs, must be manually entered into accounting, or billing systems. Staff members typically read and type the invoice details such as invoice number, vendor information, date, items, quantity, price, taxes, and total amount. This method is highly susceptible to human errors such as typographical mistakes, missing fields, duplicate entries, or incorrect calculations.

ii. **Verification and Approval Process:**

After entering the invoice data, it undergoes a verification process where the details are cross-checked with purchase orders, delivery receipts, or previous records. This step also relies heavily on manual inspection, making it time-consuming and inefficient**,** especially when the invoice volume is high.

iii. **Limited Automation and Integration:**

Most existing systems have limited automation features and often lack integration with CRM systems. This means that after processing an invoice, staff have to manually update customer or client accounts, track payments, and generate reports. As a result, organizations may experience delays in updating

records, following up on payments, or reconciling accounts.

iv. **High Error Rate and Redundancy:**

Since the process depends on manual entry, errors such as incorrect amounts, tax miscalculations, or misclassification of vendors are common. Redundant efforts like rechecking and re-entering data consume additional resources and delay the workflow.

v. **Limited Reporting and Analysis:**

Traditional invoice processing systems provide only basic reporting, making it difficult for management to gain actionable insights. Analysis of invoice trends, vendor performance, cash flow, and outstanding payments is slow and inefficient, affecting strategic decision-making.

vi. **Resource-Intensive and Non-Scalable:**

Processing large volumes of invoices requires significant manpower, which increases operational costs. Additionally, the manual system struggles to scale when business operations expand or invoice volume grows significantly.

3.1.1 Drawbacks

Manual invoice processing presents several challenges the efficiency and accuracy in financial operations. The process is highly time-consuming, leading to delays in invoice approvals and payments. The lack of automation results in repetitive manual tasks and the need for continuous human involvement, especially since there is no direct integration with the CRM system, requiring manual updates. As the volume of invoices increases, scalability becomes difficult, and operational costs rise due to heavy reliance on manpower.

3.2 PROPOSED SYSTEM

The proposed system introduces an AI-based automated solution to overcome the limitations of the existing manual invoice processing system. It integrates Optical Character Recognition (OCR), Machine Learning (ML), and CRM automation to streamline the entire workflow, reduce errors, and improve efficiency.

i. **Automated Data Extraction**

Invoices received in any digital format (PDF, scanned images) are automatically processed using OCR and AI algorithms. The Key invoice details such as invoice number, vendor information, date, line items, quantities, taxes, and total amounts are extracted accurately without manual entry. AI models can recognize different invoice formats from multiple vendors, eliminating the need for template-based processing.

ii. **Validation and Error Detection**

Extracted data is automatically validated against predefined rules, previous invoice records, purchase orders, and vendor details. The system detects inconsistencies, missing fields, duplicate invoices, or incorrect amounts and flags them for review. This reduces human errors and ensures accuracy and reliability.

iii. **Integration with CRM Systems**

Invoice details are automatically updated in the CRM system, reflecting client or customer accounts in real-time.

iv. **Automated Approval Workflow**

The system can implement rule-based approval workflows, sending notifications to authorized person for review only if anomalies are detected.

Reduces manual verification effort and speeds up invoice approvals.

v. **Analytics and Reporting**

Generates real-time reports on invoice status, vendor performance, pending payments, and cash flow. AI-based insights help management make faster, data-driven decisions**.**

vi. **Scalability and Efficiency**

Capable of processing hundreds or thousands of invoices simultaneously without additional manpower. Significantly reduces processing time and operational costs while increasing accuracy.

vii. **Error Handling and Exception Management**

Any anomalies or errors detected by the AI are flagged for human review, ensuring a combination of automation and human oversight for critical cases.

3.2.1 Advantages

- **Automation of Invoice Processing** - Eliminates manual data entry using AI and OCR, saving time and effort.
- **Real-Time Integration** - Invoice data is instantly updated in CRM systems without manual intervention.
- **Faster Approval Workflow** - Rule-based automated verification speeds up approvals and reduces delays.
- **Cost-Effective** - Cuts down manpower requirements, lowering operational costs.
- **Scalability** - Capable of handling large volumes of invoices efficiently as the business grows.
- **Enhanced Analytics & Reporting** - Provides real-time insights on cash

flow, vendor performance, and pending payments.

- **Error Detection & Exception Handling** - Flags anomalies and duplicates automatically, ensuring reliable processing.

3.3 Comparison Table

Aspect	Existing System	Proposed System
Data Entry	Manual entry of invoice details	Automated extraction using AI/OCR from digital invoices
Time Efficiency	Time-consuming, delays in processing	Fast processing with real-time analysis and validation
Error Rate	High, prone to human errors	Minimal errors due to AI validation and data verification
Integration with CRM	Not integrated, manual updates required	Direct integration, automatic updates to CRM systems
Scalability	Limited, difficult to handle large volumes of invoices	Highly scalable, can process hundreds/thousands of invoices
Analytics & Reporting	Limited insights and reports	Advanced analytics and insights for decision-making
Approval & Verification	Manual verification by staff	Automated validation against predefined rules and past records
Cost & Labor	High labor costs due to manual handling	Reduced labor costs, optimized resource utilization
Error Handling	Human-dependent correction	AI detects anomalies and flags for review

Table. 3.3 Comparison Table

CHAPTER 4
SOFTWARE REQUIREMENTS ANALYSIS

4.1 Hardware Requirements

4.1.1 Development Environment:

- **Processor:** Intel i5/i7 (8th Gen or higher) or AMD equivalent
- **RAM:** Minimum 8 GB (16 GB recommended for AI model training)
- **Storage:** 256 GB SSD (512 GB preferred)
- **GPU:** NVIDIA GPU (CUDA enabled, e.g., GTX 1660 / RTX 2060 or higher) for faster AI/ML processing

4.1.2 Deployment/Server Environment:

- **Processor:** Intel Xeon / AMD EPYC (multi-core)
- **RAM:** Minimum 16 GB (32 GB+ for handling large volumes of invoices)
- **Storage:** 1 TB SSD or cloud storage (AWS S3, Google Cloud Storage, Azure Blob)
- **GPU:** NVIDIA Tesla / RTX Series GPU (for AI inference at scale, optional if using cloud AI services)
- **Network:** High-speed internet connection (for API calls and cloud CRM integration)

4.2 Software Requirements

- **Operating System: Windows 10 or 11**
- **Development Frameworks and Libraries**:
  - Python (for AI/ML, OCR, backend logic)
  - TensorFlow / PyTorch (for machine learning models)
  - OpenCV / Tesseract OCR (for text extraction from invoices)
  - Flask / Django / FastAPI (for backend services & APIs)
  - ReactJS / Angular (for front-end user interface)
- **Database Management System (DBMS)**: MySQL/PostgreSQL
- **Text Editor or Integrated Development Environment (IDE)**: VS Code
- **Web Browser**: Google Chrome, Mozilla Firefox, or Safari.

4.3 Software Description

4.3.1 AI/Machine Learning Libraries:

i. Tensor Flow

TensorFlow is an open-source machine learning and deep learning framework developed by the Google Brain Team in 2015. It is designed to simplify the process of building, training, and deploying artificial intelligence models. TensorFlow works on the concept of computational graphs**,** where data flows in the form of multi-dimensional arrays called tensors, making it efficient to run on CPUs, GPUs, and TPUs. The library provides a wide range of tools, including TensorFlow Core for low-level operations, Keras API for high-level model building, TensorFlow Lite for

mobile devices and TensorFlow Extended (TFX) for production pipelines. Its scalability allows developers to train models on small datasets and later expand to large scale enterprise level applications.

ii. PyTorch

PyTorch is an open-source machine learning and deep learning library developed by Facebook's AI Research Lab (FAIR) in 2016. It has rapidly become one of the most popular frameworks in AI research and industry due to its simplicity, flexibility, and dynamic computation graph support. Unlike TensorFlow's early static graph approach, PyTorch allows developers to build and modify models on the fly, making experimentation and debugging much easier. PyTorch is written in Python and C++, which gives it both usability and high performance. It supports tensor computation with GPU acceleration and provides strong tools for building neural networks, optimization algorithms, and deep learning pipelines.

iii. Scikit-learn

Scikit-learn is one of the most popular open-source machine learning libraries for Python. It was initially developed by David Cournapeau as part of the Google Summer of Code project in 2007, and since then, it has grown into a powerful and widely used library maintained by a large open-source community. Built on top of NumPy, SciPy, and Matplotlib, Scikit-learn provides a wide range of machine learning algorithms and tools for data preprocessing, classification, clustering, dimensionality reduction, and model evaluation. Unlike deep learning frameworks such as TensorFlow or PyTorch, Scikit-learn focuses primarily on traditional machine learning techniques and is highly efficient for small to medium sized datasets.

4.3.2 OCR & Image Processing

i. Tesseract OCR (pytesseract)

Tesseract OCR is one of the most widely used open-source Optical Character Recognition (OCR) engines. Originally developed by HP Labs and later sponsored by Google, Tesseract has become a standard tool for extracting text from images and scanned documents. It is highly efficient in recognizing printed text in multiple languages, making it suitable for document digitization, automated data entry, and text-based AI applications. It is a Python wrapper for Tesseract OCR that allows developers to integrate Tesseract's capabilities directly into Python programs. It provides a simple interface to extract text from relevant sources, making it highly useful for projects that require document understanding and automation.

ii. Pdf2image

pdf2image and PyMuPDF (fitz) are popular Python libraries used for converting PDF documents into images, which can then be processed for text extraction or image analysis. Many invoices and financial documents are often shared in PDF format, making it essential to convert them into a format that OCR engines like Tesseract can read effectively. It is a lightweight Python library that converts PDF pages into high-quality images. PyMuPDF (fitz) is an advanced library for reading, rendering, and manipulating PDFs and other document types. It provides precise control over page rendering, text extraction, annotations, and images, making it ideal for complex document workflows. The libraries are widely used in invoice processing, document automation, and AI-driven text extraction systems, providing the first step in converting digital documents into machine readable formats.

4.3.3 Data Handling & Preprocessing

i. Pandas

Pandas is an open-source Python library designed for data manipulation and analysis. It was developed by Wes McKinney in 2008 and has since become a standard tool for handling tabular and time-series data. Pandas provides data structures like Series and DataFrame, which are ideal for organizing invoice data such as vendor names, invoice numbers, dates, and amounts.

ii. NumPy

NumPy (Numerical Python) is a foundational library for numerical computing in Python. Developed in 2005, NumPy provides high-performance multidimensional arrays (ndarrays) and tools for performing mathematical and statistical operations. In invoice processing, numerical operations such as totals, averages, or anomaly scoring are performed using NumPy arrays.

4.3.4 Backend and API

i. Flask

Flask is a lightweight, open-source web framework for Python that allows developers to build web applications and APIs quickly and efficiently. Flask follows a minimalistic approach, providing only the essential components required for web development while giving developers the flexibility to extend functionality through external libraries and plugins. Flask is often referred to as a “micro-framework” because it does not impose a particular project structure or include unnecessary tools by default. This makes it ideal for small to medium-sized applications or projects where custom solutions are needed.

ii. Requests

Requests is a widely used Python library for sending HTTP/HTTPS requests in a simple and human-readable way. It was created by Kenneth Reitz in 2011 and has become the de facto standard for HTTP client programming in Python. The library abstracts the complexities of network communication, making it easy to send GET, POST, PUT, DELETE, and other HTTP requests. Requests is designed to be simple, reliable, and intuitive, allowing developers to interact with web services, REST APIs, and external systems without writing verbose or complicated code.

iii. SQLAlchemy

SQLAlchemy is a powerful Python SQL toolkit and Object-Relational Mapping (ORM) library**.** It was created by Michael Bayer in 2005 and has since become a widely used standard for database management in Python applications. SQLAlchemy allows developers to interact with relational databases like MySQL, PostgreSQL, SQLite, and Oracle using Python objects instead of writing raw SQL queries, making database operations easier, safer, and more maintainable. SQLAlchemy consists of two main components:

1. **Core** – Provides a SQL expression language to write database queries programmatically with Python syntax.
2. **ORM (Object-Relational Mapper)** – Maps Python classes to database tables, allowing developers to work with objects instead of raw tables and rows.

4.3.5 Data Base libraries

i. MySQL Connector

MySQL Connector/Python is an official MySQL driver provided by Oracle

to connect Python applications with MySQL databases. It allows developers to execute SQL queries, manage transactions, and handle database connections directly from Python.

ii. PostgreSQL

PostgreSQL is an advanced open-source relational database system known for its robustness, scalability, and compliance with SQL standards. psycopg2 is a popular Python library used to connect Python applications with PostgreSQL databases. It provides support for transactions, prepared statements, and advanced features like JSON storage, concurrency, and full-text search.

4.3.6 Reporting & Visualization

i. Matplotlib

Matplotlib is a comprehensive Python library for data visualization. It was originally developed by John D. Hunter in 2003 and has since become one of the most widely used visualization libraries in the Python ecosystem. Matplotlib provides 2D plotting capabilities to generate high-quality graphs, charts, and figures from numerical or structured data. In projects like the AI-Based Invoice Analyzer, visualizing data is crucial for understanding patterns, monitoring workflow efficiency, and presenting insights to users or management. Matplotlib enables developers to convert complex data into clear and interpretable visual formats.

ii. Plotly

Plotly is a modern Python library for interactive data visualization. It enables developers to create dynamic, web-ready charts, dashboards, and analytics applications. Plotly supports a wide range of plots, including line

charts, bar charts, scatter plots, pie charts, heatmaps, 3D plots, and geographic maps. Unlike static visualization libraries, Plotly produces interactive plots that allow zooming, panning, hovering, and filtering. Plotly is especially important in projects like the AI-Based Invoice Analyzer, where stakeholders require real-time insights and interactive reporting dashboards to monitor invoice trends, AI predictions, and financial analytics.

iii. ReportLab

ReportLab is a powerful Python library for generating PDFs and complex documents programmatically. It was created by Andy Robinson in 2000 and is widely used for automated report generation. ReportLab allows developers to create dynamic, professional-quality PDF files with text, images, tables, charts, and custom formatting, all directly from Python code. In projects like the AI-Based Invoice Analyzer, ReportLab is essential for generating automated invoice reports, summaries, and analytics outputs in a format that is easy to share, print, and archive.

CHAPTER 5
SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The System Architecture begins with the Data Ingestion phase, where invoices are uploaded for processing. The Data Extraction module, using (pytesseract) extracts relevant information from the invoices. The extracted data is Validated through tools like Pandas and NumPy to ensure accuracy before being stored in a Database like as MySQL or PostgreSQL. The CRM System accesses this validated data to automate customer relationship processes and improve efficiency. Finally, the system performs Report Analysis and generates Upsell Suggestions using AI and machine learning frameworks to provide valuable business insights and recommendations.

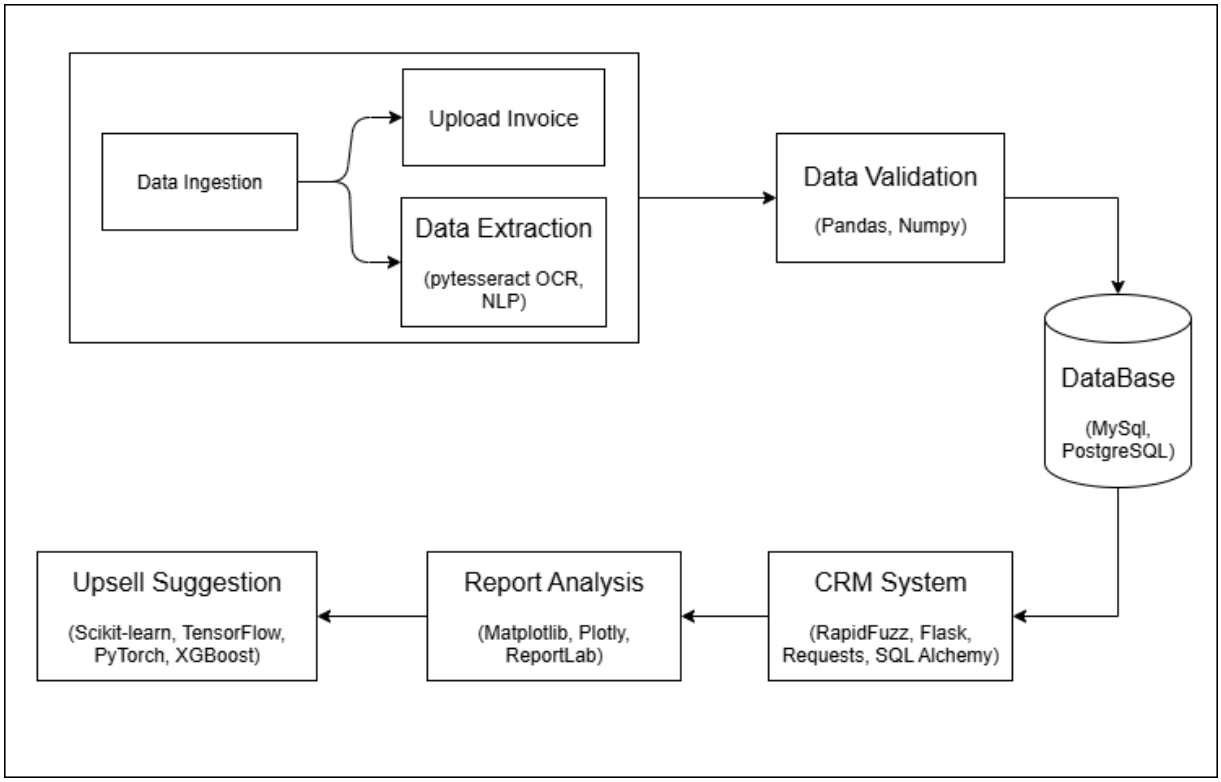

Fig. 5.1. System Architecture

5.2 ER Diagram

The ER Diagram represents the Invoice Model structure, detailing how various invoice components such as company details, client information, payment info, product data, and identification numbers are interconnected. It defines the organized schema for extracting, validating, and storing all invoice-related fields systematically for automated processing.

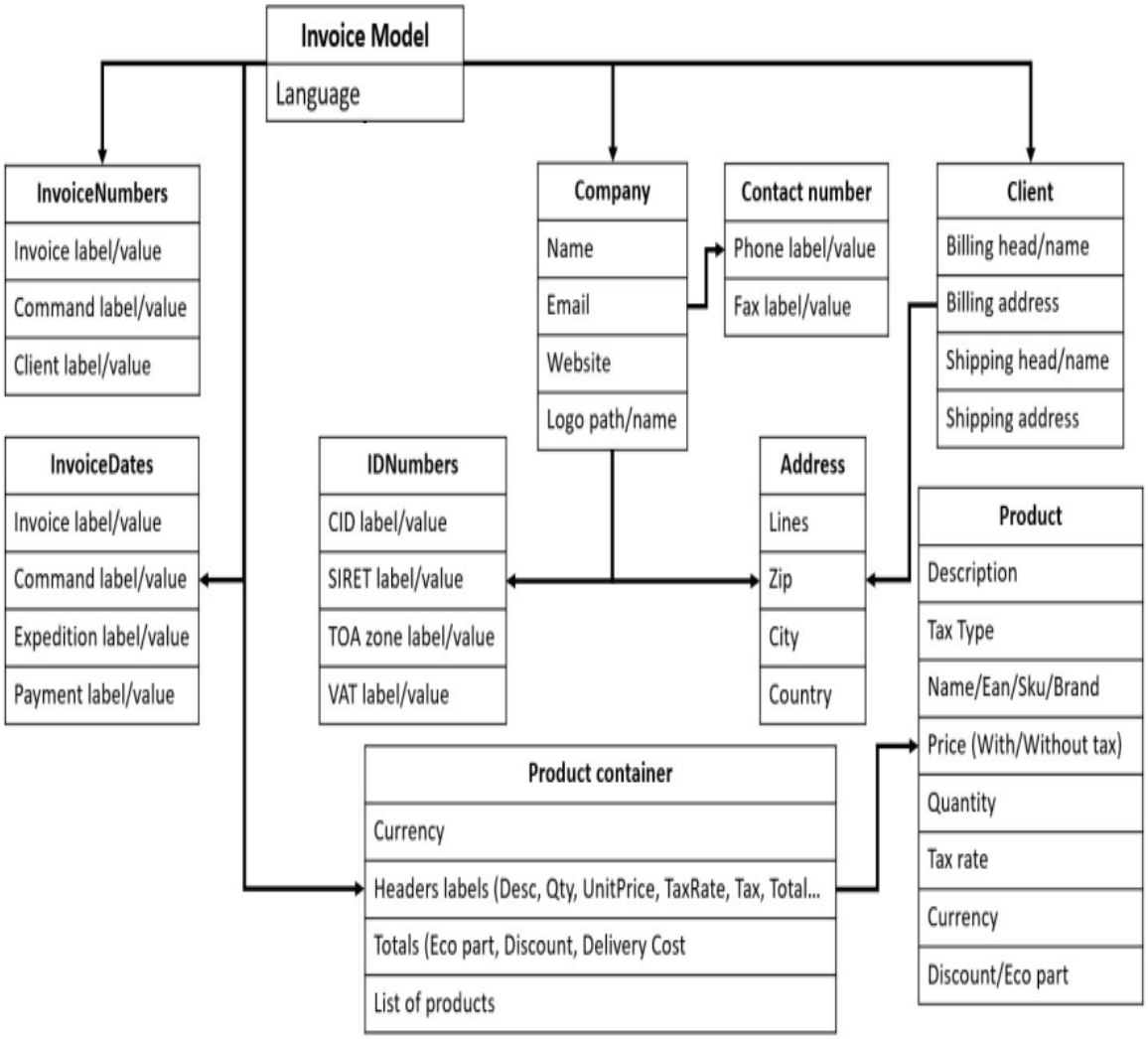

Fig. 5.2 ER Diagram

5.3 OCR Extraction Template

The OCR Extraction Template shows the layout structure of an invoice document, organized into sections like header, product table, product footer, and global footer. It defines how elements such as company details, billing and shipping addresses, product information, VAT, and totals are positioned in relation to each other for systematic invoice generation.

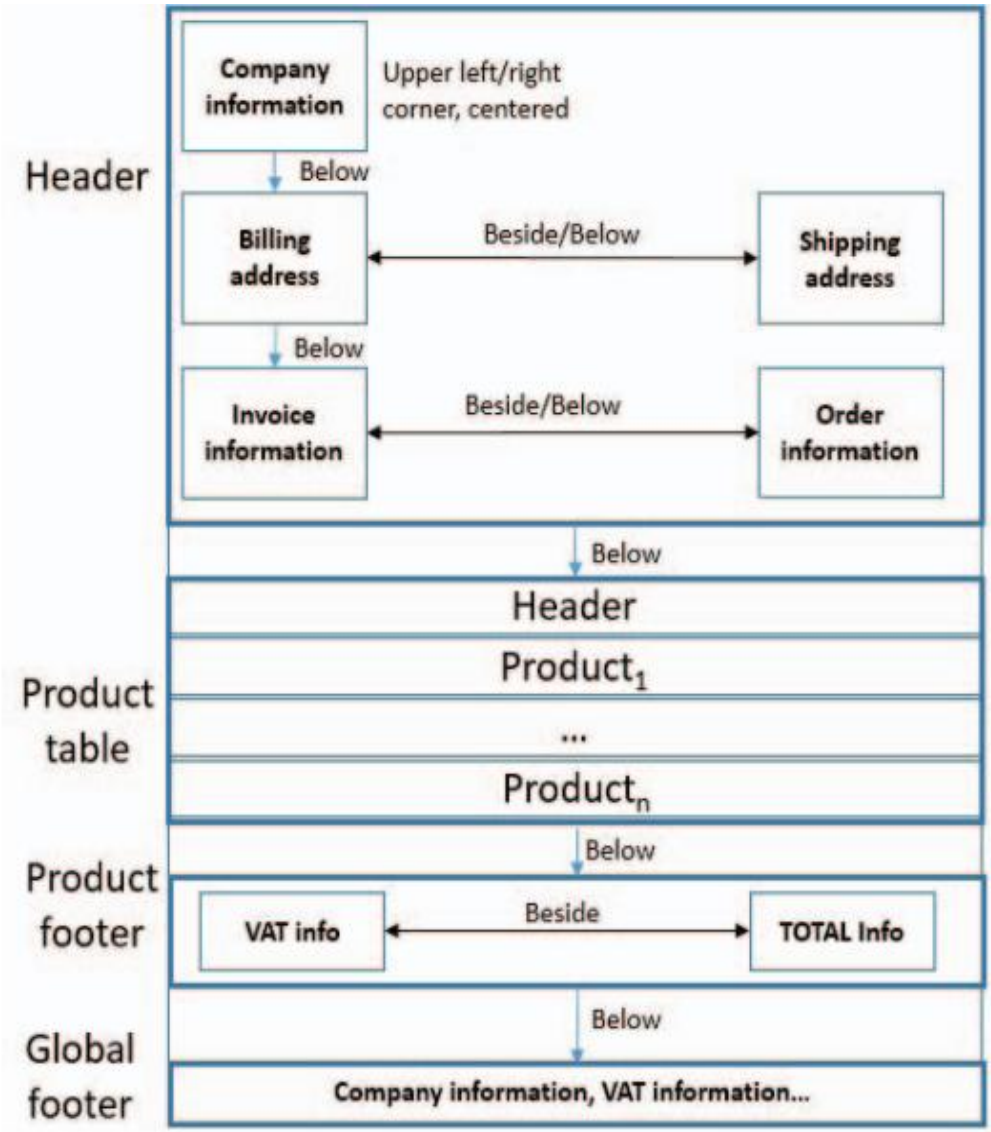

Fig. 5.3 OCR Extraction Template

5.4 Database Schema

The Database Schema is defines as the structured organization of all data stored in the system. It includes key tables such as Invoices, Customers, Vendors, and Transactions, each linked through primary and foreign keys. The schema ensures smooth data flow between modules like invoice upload, analysis, and CRM matching. It maintains data integrity, supports relational queries, and enables fast retrieval of invoice and customer information. Overall, it provides a robust foundation for efficient storage, access, and integration of financial data.

1. **Invoice Table:**

Column Name	Data Type	Description
Invoice_id	INT (PK)	Unique identifier for the invoice
Vendor_id	INT (FK)	Linked vendor ID
Invoice_number	VARCHAR	Invoice number
Invoice_date	DATE	Date of the invoice
Total_amount	DECIMAL	Total amount including taxes
status	VARCHAR	Processing Status (Pending / Approved / Rejected)

Table.5.4.1. Invoice Table

2. **Vendor Table:**

Colunm Name	Data Type	Description
vendor_id	INT (PK)	Unique vendor ID
Vendor_name	VARCHAR	Vendor name
Contact_info	VARCHAR	Email/Phone

Table.5.4.2. Vendor Table

3. **Line_Item Table:**

Column Name	Data Type	Description
Item_id	INT (PK)	Unique line_item ID
Invoice_id	INT (FK)	Associated invoice
Description	VARCHAR	Product/Service description
Quantity	INT	Quantity
Unit_price	DECIMAL	Price per unit
total	DECIMAL	Total price for this line item

Table.5.4.3. List_Item Table

4. **CRM_Mapping Table:**

Colunm Name	Data Type	Description
mapping_id	INT (PK)	Unique mapping ID
invoice_id	INT (FK)	Linked invoice
crm_record_id	VARCHAR	CRM record ID for validation
validation_status	VARCHAR	Status of CRM data validation

Table.5.4.4. CRM Mapping Table

5. **Notifications Table:**

Colunm Name	Data Type	Description
notification_id	INT (PK)	Unique notification ID
invoice_id	INT (FK)	Linked invoice
message	TEXT	Notification message
status	VARCHAR	Sent/Delivered/Pending

Table.5.4.5. Notifications Table

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 SYSTEM IMPLEMENTAION

System implementation involves translating the system design into functional software components and deploying them in a production environment. Here's a step-by-step guide:

i. **Setting Up Development Environment:**
   - Install necessary development tools and software, including Python, OCR, pytesserest, OpenCV, SQLAlchemy, Flask, Requests, Pandas, NumPy, Matplotlib, ReportLab and other required libraries.

ii. **Backend Development:**
   - Create a Project Structure and add the relevant files to it.
   - Develop the RESTful APIs for invoice upload where it accepts the files from frontend and Data Retrieval API to process invoice data and for the CRM Integration API which sends the invoice summaries or reports.
   - Integrate AI Models to load trained TensorFlow or Scikit-learn Models for the validation of the invoice fields, detect anomalies and classify invoices.
   - Implement the Security and Error Handling for the Validation of the inputs to prevent injection attacks, and add exception handling for API Calls, file uploads, and AI Predictions.

iii. **Frontend Development:**

- Create a Frontend Project Structure and add the relevant files to it. In the Invoice Upload Interface, Drag-and-Drop or browse functionality are used for invoices. And display the upload progress and status.
- It gives the Dashboard and Visualization by using Plotly, Matplotlib or Seaborn for interactive Charts and it shows the Vendor-wise summaries, anomaly alerts and trends.

iv. **Database Setup**:

- Set up MySQL database instance and configure connection settings in the Flask application.
- Create database tables, indexes, and relationships based on defined schemas and models.
- Configure the ORM with SQLAlchemy which Maps the Python Classes to database tables and it enable CRUD operations via Python Objects.
- It implement the connection Pooling and Transactions which ensure high performance for multiple simultaneous uploads, and it maintains the data consistency and rollback on failures**.**

v. **Integration & Testing:**

- It integrates the frontend with Backend APIs by using HTTP requests (via Axios or Fetch in React) to communicate with Flask/FastAPI.
- Integrate AI Models with OCR & Database by ensuring smooth flow by uploading to Preprocessing to OCR to AI to Database to Visualization to CRM.

By testing the System Functionality the unit testing for modules and integration testing for end-to-end invoice processing.

6.2 MODULES DESCRIPTION

i. Invoice Upload Module:

The Invoice Upload Module is the entry point of the system where all invoices are submitted for processing. Users or connected CRM systems can upload invoices in formats such as PDF, PNG, or JPEG through a user-friendly frontend interface. The module first performs file validation, checking for correct formats, file size limits, and file integrity to prevent errors in subsequent stages. It supports batch uploading, allowing multiple invoices to be uploaded simultaneously, which is essential for businesses dealing with large volumes of invoices. The module also maintains metadata for each invoice, including the uploader ID, upload timestamp, and file name, which helps in tracking and auditing. Once validation is successful, the invoice files are stored in a structured temporary storage system categorized by vendor or date. Security measures like access control and input sanitization are implemented to ensure sensitive financial data is protected. After storage, the files are sent automatically to the image preprocessing module for further analysis. This module ensures efficient, secure, and organized ingestion of invoices, forming the foundation for accurate OCR extraction and AI Processing.

ii. Image Preprocessing and OCR Module

The Image Preprocessing and OCR Module prepares the uploaded invoices for accurate text extraction. Initially, the uploaded images are passed through OpenCV preprocessing techniques**,** which include grayscale conversion, noise

removal, thresholding, and edge enhancement**.** These steps improve image clarity, reduce distortions, and ensure better OCR accuracy. After preprocessing, Tesseract OCR (pytesseract) is used to extract textual content from the invoices. OCR converts the visual text in images or scanned PDFs into machine-readable text, capturing key invoice fields such as invoice number, date, vendor, name, line items, total amount, and tax details. The module also includes error correction routines, such as detecting and fixing common OCR misreads, removing extraneous characters, and handling multi-line text fields. Extracted text is then sent to the data structuring module, where it will be cleaned and organized. By combining image enhancement and OCR, this module ensures that textual data is captured accurately and ready for automated analysis, which is critical for downstream AI validation and anomaly detection.

iii. Data Structuring and Cleaning Module:

Once OCR extracts the raw text, the Data Structuring and Cleaning Module converts it into a well-organized, structured format. Using Pandas and NumPy, the module parses text into predefined fields such as invoice number, invoice date, vendor details, itemized amounts, tax, and total. It applies regular expressions and string processing techniques to extract numbers, dates, and text accurately, correcting any inconsistencies from the OCR output. Missing values are detected and handled, duplicates are removed, and data types are standardized for seamless integration into AI models and the database. Metadata from the upload stage is also included in the structured data for tracking. This module ensures that all invoice data is accurate, consistent, and ready for AI analysis, minimizing errors in classification and anomaly

detection. By producing clean, tabular data, this module forms the backbone of the automated invoice processing workflow.

iv. AI Processing Module:

The AI Processing Module analyzes structured invoice data to validate fields, classify invoices, and detect anomalies. Using TensorFlow or Scikit-learn, the module runs classification models to categorize invoices by vendor, invoice type, or payment status. Anomaly detection models identify irregularities, such as mismatched amounts, duplicate invoices, or unusual tax values. The AI model also validated extracted fields against historical patterns and business rules, ensuring consistency and accuracy. Each invoice is assigned a confidence flagged anomalies, classifications, and validation results, are appended to the structured dataset. This module plays a critical role in reducing human error, improving financial compliance, and highlighting invoices that require attention, thus making the system highly intelligent and reliable.

v. Database Module:

The Database Module handles the storage and management of all structured data and AI outputs**.** Using SQLAlchemy ORM with MySQL or PostgreSQL, structured invoice data, AI results, metadata, and reports are stored in relational tables. Key tables include Invoices, Vendors, AI_Results, Users, and Reports, with relationships defined to maintain integrity. The module supports CRUD operations, allowing retrieval, updating, and deletion of invoice records. Transactions ensure atomic operations to maintain consistency, while connection pooling improves performance for simultaneous uploads. Secure access control ensures that only authorized users or services can query or

modify the database. This module provides reliable, scalable, and efficient data storage, enabling downstream modules like visualization, reporting, and CRM integration to access accurate and up-to-date information.

vi. Backend API Module:

The Backend API Module exposes the system's functionality through RESTful endpoints, enabling communication between the frontend, database, AI models, and CRM. Developed using Flask or FastAPI**,** it handles file uploads, invoice data retrieval, AI model processing, report generation, and CRM integration. Each API includes authentication, input validation, and error handling to ensure secure and reliable operations. The module coordinates the workflow, sending invoices to the preprocessing/OCR module, receiving structured data, invoking AI models, and storing outputs in the database. It also provides endpoints for the frontend dashboard to retrieve analytics and visualizations in real-time. The backend ensures smooth, automated, and scalable interaction between all system components.

vii. Visualization and Reporting Module:

The Visualization and Reporting Module generates insights and summaries from processed invoice data. Using Matplotlib, Seaborn, and Plotly, interactive dashboards display vendor-wise invoice totals, overdue invoices, trend analysis, and AI-detected anomalies. Users can filter data by vendor, date range, or invoice status to explore patterns. Reports are also generated using ReportLab, creating professional PDF summaries with tables, charts, and AI outputs. These reports can be downloaded or automatically sent to the CRM system. The module enhances decision-making, transparency, and operational efficiency, providing a clear visual representation of all processed invoices and analytical results.

viii. CRM Integration Module:

The CRM Integration Module ensures that processed invoice data and reports are synchronized with the organization's CRM system. Using the Requests library and REST APIs, the module sends structured invoice data, AI results, and PDF reports to the CRM in real-time. It ensures that finance teams and management can monitor invoice processing, payments, and anomalies directly within the CRM. Automated integration reduces manual effort, improves accuracy, and enables centralized tracking of invoices and financial data across the organization. This module completes the workflow by linking all automated processing to enterprise operations.

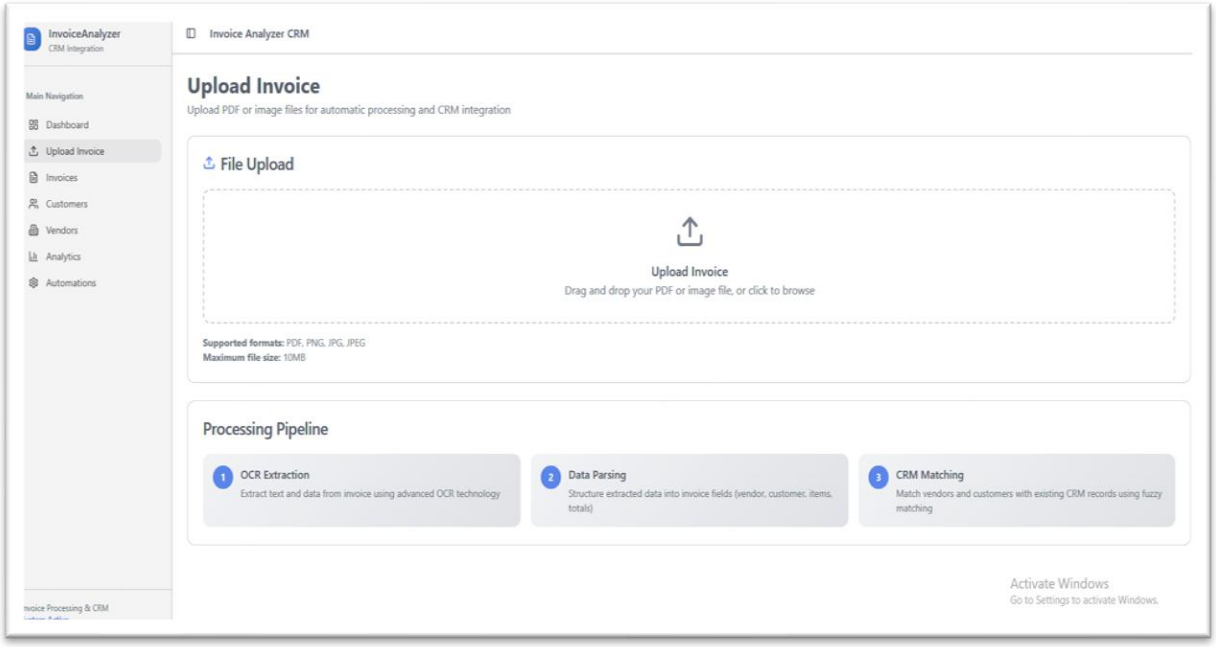

Fig. 6.1 Invoice Upload Page

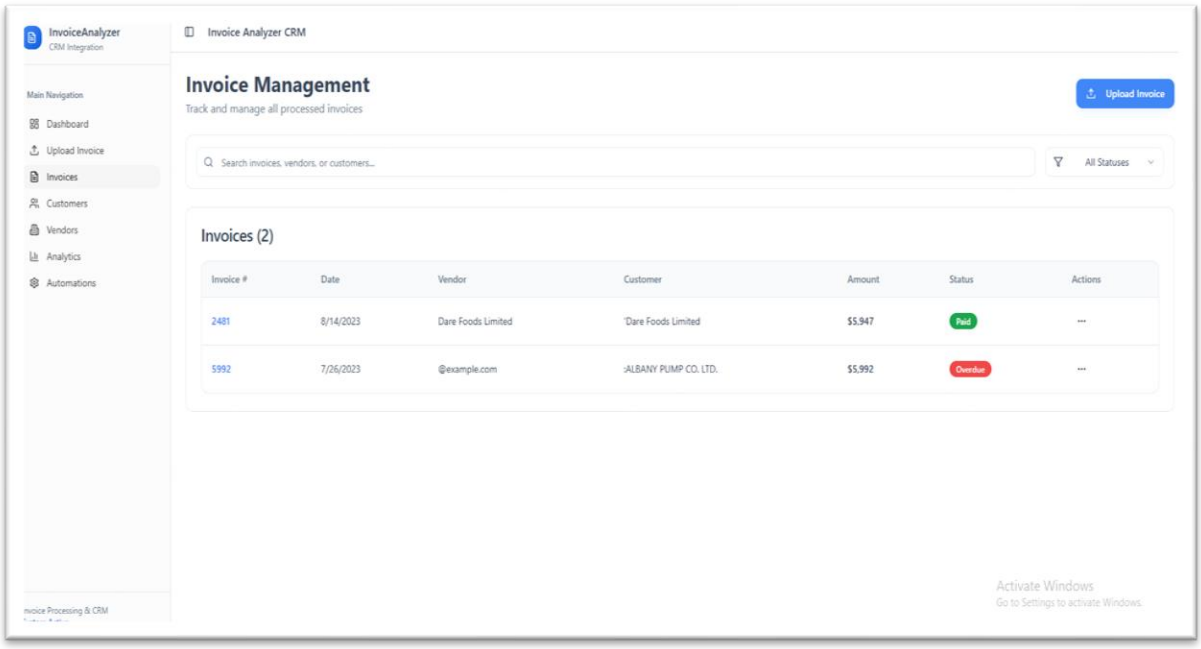

Fig. 6.2 Invoice Extracted and Display Page

CHAPTER 7
RESULT AND ANALYSIS

7.1 Accuracy, Precision, Recall, and F1-Score

The performance of the proposed AI-Based Invoice Analyzer was evaluated using standard classification metrics: Accuracy, Precision, Recall, and F1-Score**.** These metrics help measure the system's ability to correctly extract and validate invoice fields such as invoice number, date, vendor details, and total amount.

- **Accuracy**: It measures the proportion of correctly predicted invoice fields (correct extractions and matches) out of all invoice fields processed. In invoice analysis, accuracy shows how well the system overall recognizes fields like *invoice number, vendor name, date, line items, and total amount*. **(Eg.)** If the system processed 1000 invoice fields and 969 were correctly extracted, the accuracy = **96.9%**.

- **Precision**: It indicates the percentage of correctly extracted invoice fields among all fields the system predicted as correct.
    - **Formula**: $Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$
    - **Relevance**: Invoices may have noise (scanned logos, tables, or irrelevant text). Precision ensures that when the system claims a field (e.g., total amount = ₹45,000), it is correct and not wrongly picked. **(Eg.)** If the system predicted 100 "invoice numbers" but 96 were correct and 4 were wrong, precision = **96%**.

- **Recall**: It measures the percentage of correctly extracted invoice fields out of all the actual fields present in the invoices.

- **Formula**: $Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$
- **Relevance**: Ensures that the system does not miss any critical information n (like missing line items or invoice dates). (Eg.) If an invoice had 50 line items, and the system extracted 47 correctly but missed 3, recall = **94%.**

- **F1-Score:** F1-Score is the harmonic mean of precision and recall, balancing both correctness and completeness.
  - **Formula**: $F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$
  - **Relevance**: Useful in invoice processing where both precision (avoiding wrong values) and recall (avoiding missed values) are equally important. **(Eg.)** With precision = **96%** and recall = **97%,** F1-Score = **96.5%.**

The system demonstrated high accuracy and recall, proving its effectiveness in extracting invoice data across multiple formats. Precision was slightly lower in cases where scanned invoices contained noise or poor image quality, leading to misclassification of certain fields.

7.2 Performance Evaluation Table

Metric	Invoice Number	Vendor Name	Data	Line Items	Total amount	Overall
Accuracy (%)	98.2	97.5	96.8	94.6	98.0	96.9
Precision (%)	97.9	96.7	95.4	93.1	97.2	96.1
Recall (%)	98.5	97.3	96.2	94.0	98.6	96.9
F1-Score (%)	98.2	97.0	95.8	93.5	97.9	96.5

Table.7.2. Performance Evaluation Table

Observation:

- Invoice numbers and total amounts achieved the highest accuracy as they are clearly formatted fields.
- Line items had comparatively lower performance due to variability in invoice structures and formatting.
- Overall, the System consistently achieved above 95% accuracy, proving its reliability.

7.3 Comparative Analysis with Manual Processing

To demonstrate the effectiveness of the proposed system, a comparative analysis was carried out between manual invoice processing and the AI-based automated system:

Parameter	**Manual Processing**	**AI-Based Invoice Analyzer**	**Improvement**
Average Processing Time	8-12 minutes per invoice	1-2 minutes per invoices	-80% faster
Error Rate	5-10% (data entry mistakes)	<2% (mainly due to poor scans)	-70% fewer errors
Handling Volume	Limited (20-30) Invoices/day)	Scalable (Hundreds per days	10x more capacity
Transparency	Low (manual tracking)	High (real-time dashboard)	Full Visibility
Cost Efficiency	High Labor Cost	Reduced manpower requirement	Significant savings

Table. 7.3 Comparative Analysis with Manual Processing

Discussion:

- Manual invoice processing is time-intensive, inconsistent, and error-prone.
- The proposed AI system drastically reduces processing time and improves accuracy.
- Automation ensures scalability and transparency, allowing organizations to handle bulk invoices efficiently.
- The system improves vendor relationships by ensuring timely approvals and payments.

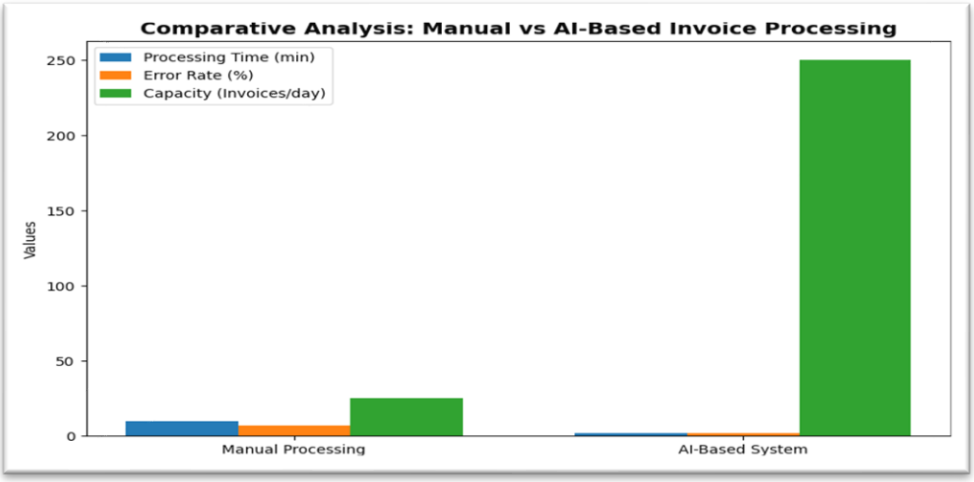

Fig. 7.3.1 Comparative Analysis

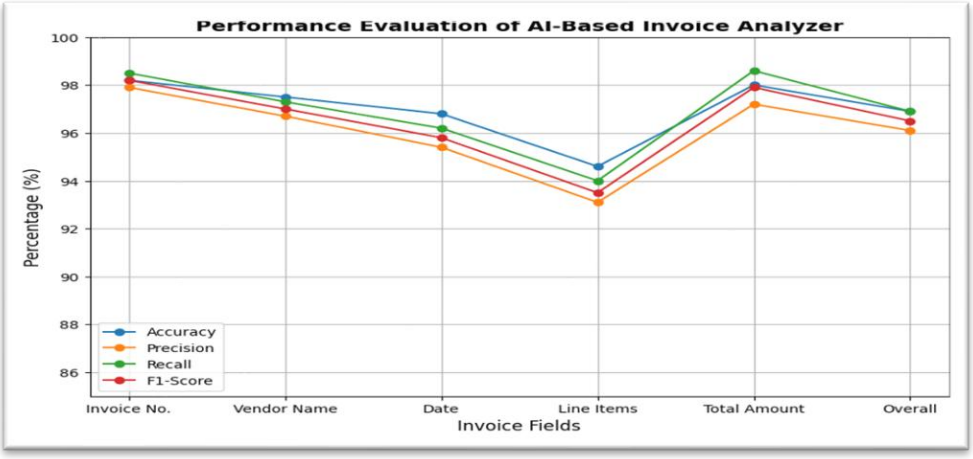

Fig. 7.3.2 Performance Evaluation

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

In Conclusion, the development AI-Based Invoice Analyzer with CRM Automation successfully demonstrates the capability of artificial intelligence to streamline invoice processing and reduce manual workload. Traditional invoice handling is time-consuming, error-prone, and requires significant human intervention for tasks such as data entry, validation, and CRM updates. By leveraging OCR (Optical Character Recognition), NLP (Natural Language Processing), and Machine Learning models, the system accurately extracts invoice fields such as invoice number, vendor details, dates, line items, and total amounts. The performance evaluation measured through Accuracy, Precision, Recall, and F1-Score shows consistently high values (above 95% for most fields), proving that the system is both reliable and efficient. Comparative analysis with manual processing further highlights the advantages of the AI-based approach in terms of faster processing time, reduced error rate, and higher daily capacity. By integrating with CRM systems, the solution ensures seamless real-time financial record updates, reduces operational costs, and enhances decision-making through analytics and reporting. Overall, the project validates the effectiveness of AI-driven automation in financial document management and sets the stage for smarter enterprise workflows.

8.2 Future Enhancement

While the current system demonstrates strong performance, there are opportunities to further improve and expand its capabilities:

- **Integration with CRM System:**
  Beyond CRM, direct integration with ERP tools (SAP, Oracle NetSuite, Tally etc.,) for end-to-end financial automation.

- **Fraud Detection Module:**
  Incorporate AI models to detect duplicate invoices, anomalies, or fraudulent billing patterns.

- **Mobile App Support:**
  Allow users to upload invoices via a mobile app (camera-based scanning), making the system more accessible.

- **Adaptive Learning & Continuous Improvement:**
  Implement self-learning models that improve accuracy as more invoices are processed over time.

- **Advanced Analytics & Dashboard:**
  Provide detailed financial insights, trends, and predictive analytics through interactive dashboards.

APPENDIX 1

SNAPSHOTS

Upload Invoice:

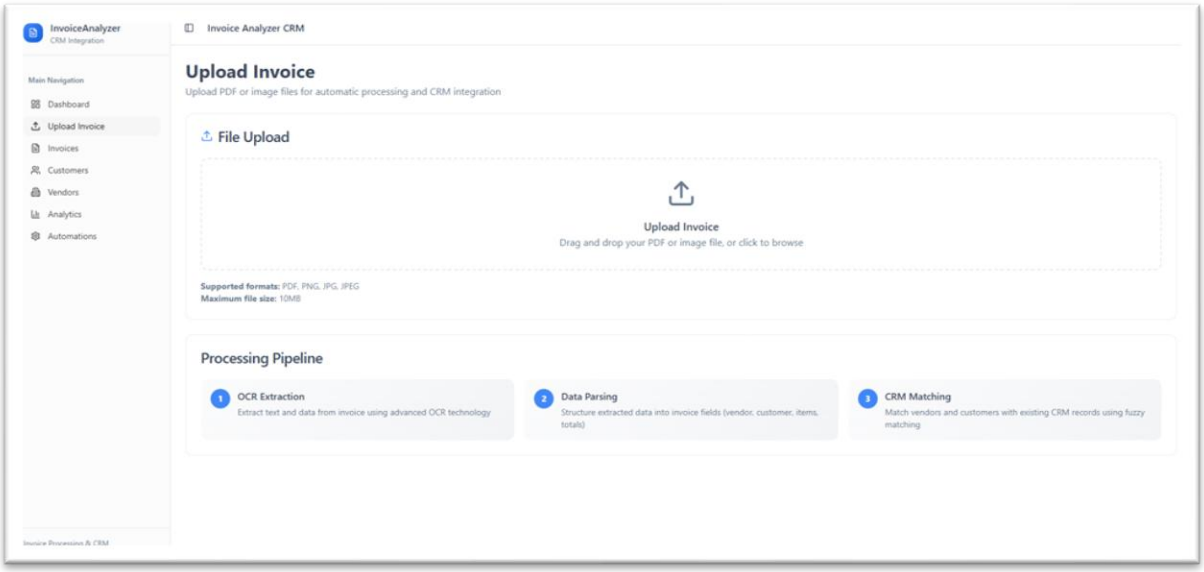

Fig. Appendix1.1 Upload Invoice

Invoice Dashboard:

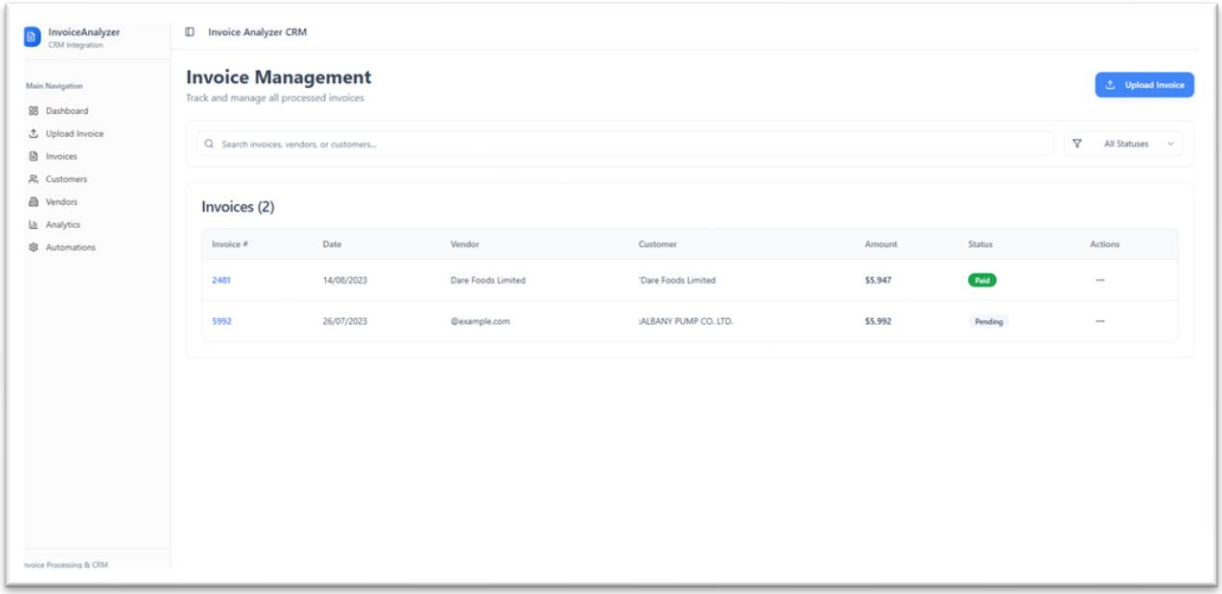

Fig. Appendix1.2 Invoice Dashboard

Customer Dashboard:

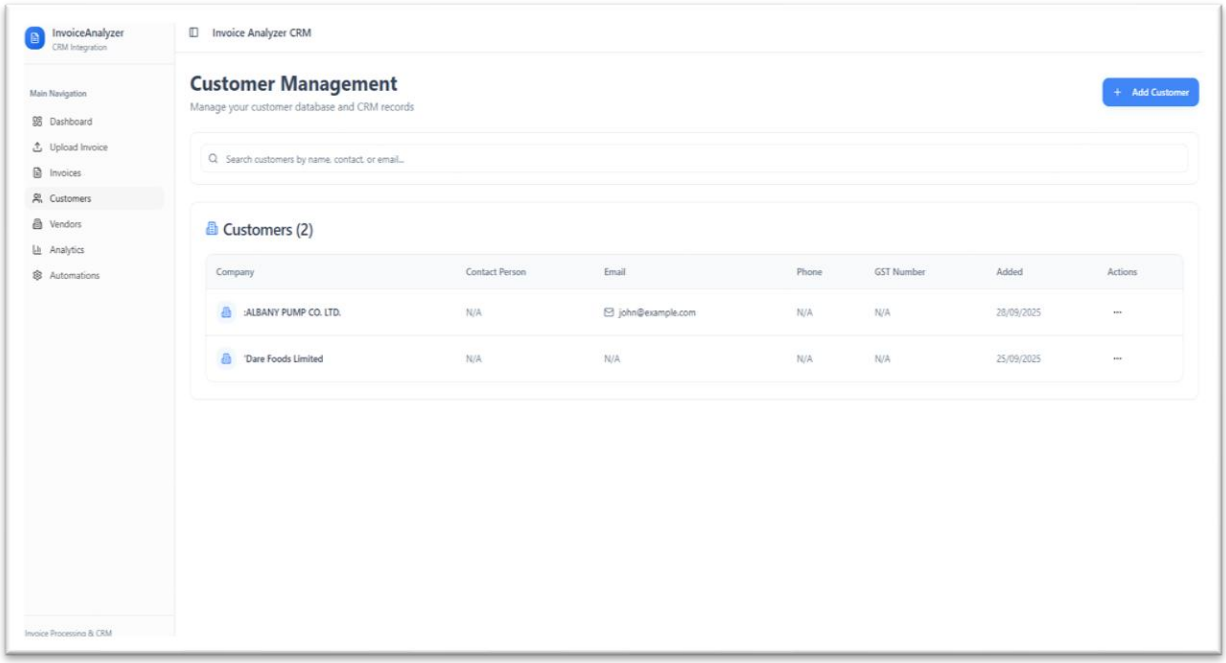

Fig. Appendix 1.3 Customer Dashboard

Vendor Dashboard:

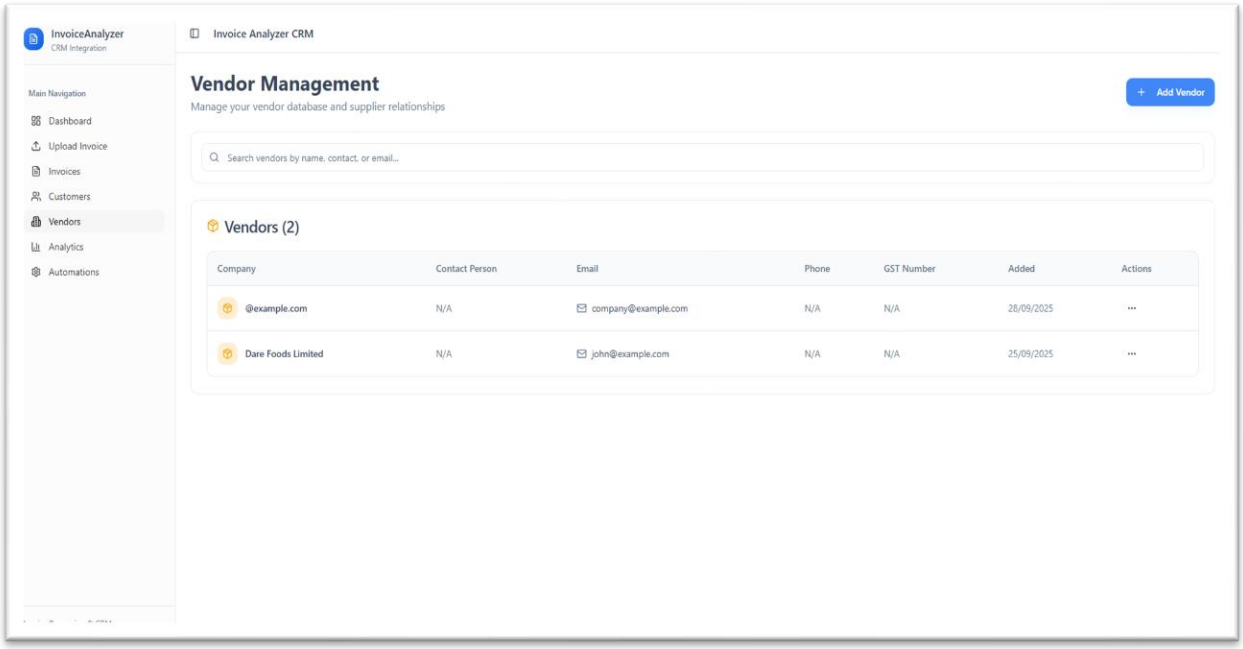

Fig. Appendix1.4 Vendor Dashboard

Analysis Dashboard:

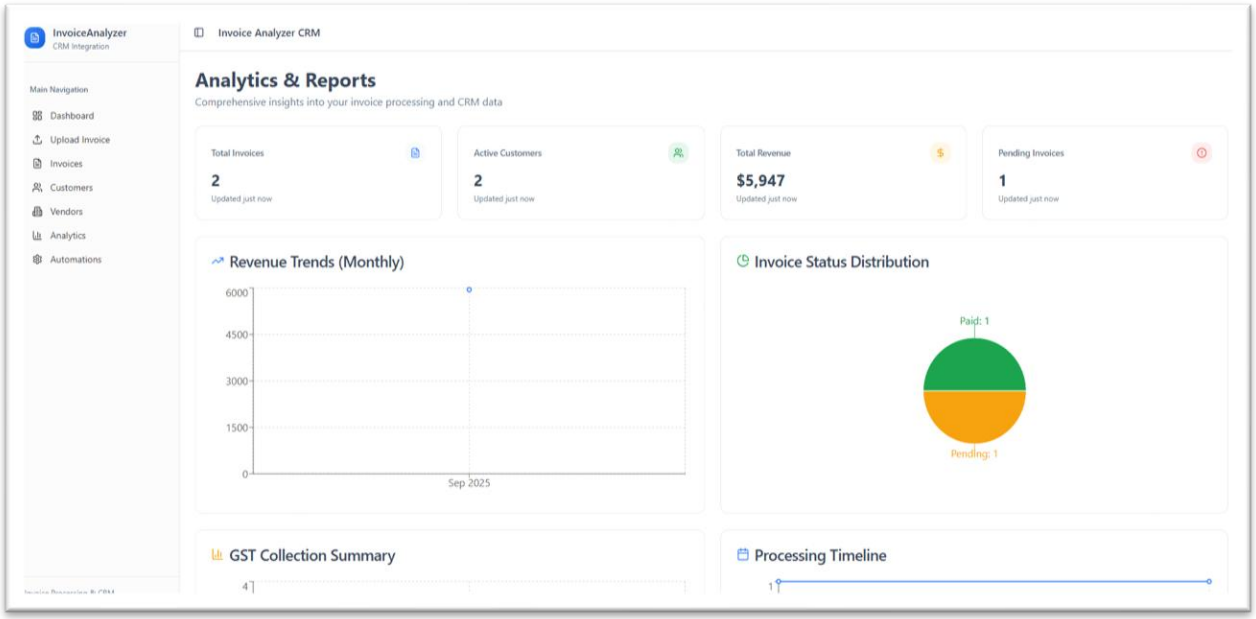

Fig. Appendix1.5 Analysis Dashboard

Upsell Suggestion Dashboard:

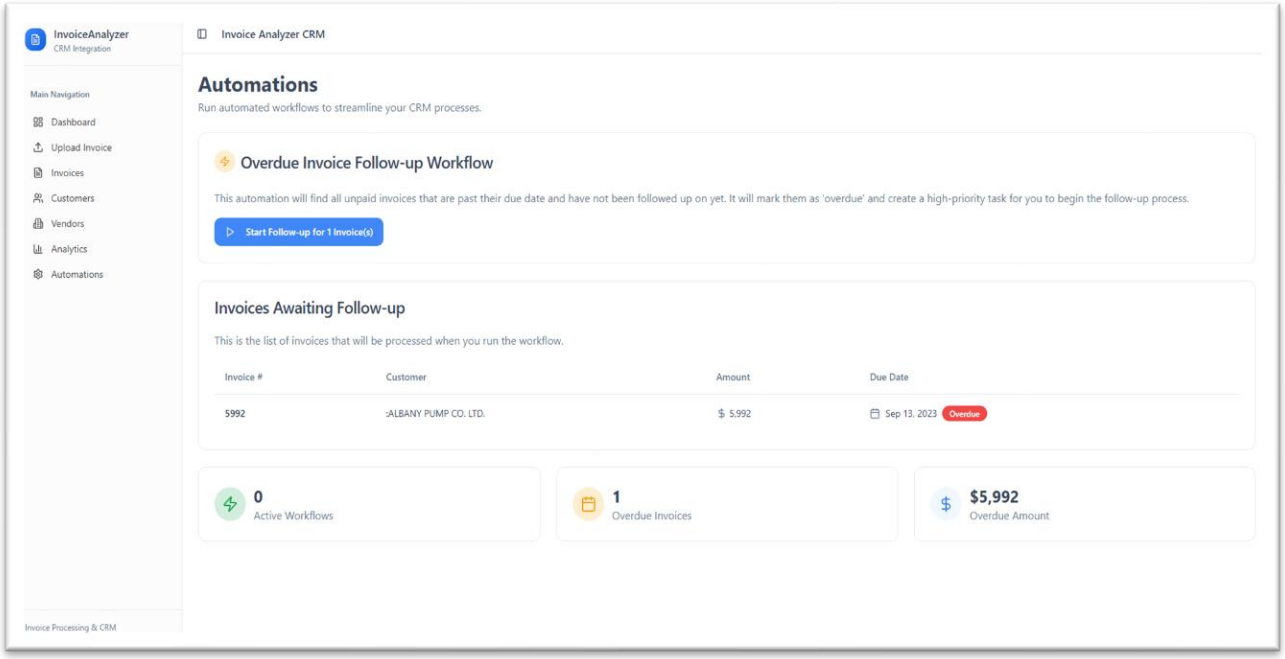

Fig. Appendix1.6 Upsell Suggestion Dashboard

APPENDIX 2

SOURCE CODE

OCR Process:

```
Tesseract.js OCR and invoice parsing utilities
// Runs entirely in the browser – no cloud OCR services are used.
import Tesseract from 'tesseract.js';
export interface OCRItem {
  name: string;
  quantity: number;
  rate: number;
  amount: number;}
export interface OCRData {
  invoice_number: string;
  total_amount: number;
  invoice_date: string;
  due_date: string;
  vendor_name: string;
  vendor_email: string;
  customer_name: string;
  customer_email: string;
  subtotal: number;
  gst_percentage: number;
  gst_amount: number;
  items: OCRItem[];}
export async function recognizeImageWithTesseract(file: File):
Promise<string> {
```

```
// Convert PDFs to a bitmap using the browser if needed is non-trivial; for now,
  // rely on Tesseract's PDF handling, which rasterizes pages internally.
  const { data } = await Tesseract.recognize(file, 'eng', {
    logger: () => {},
    tessedit_pageseg_mode: 1,
  } as any);
  return (data?.text || '').trim();}
export function parseInvoiceFromText(text: string): OCRData {
  const match = (re: RegExp) => (text.match(re)?.[1] || '').trim();
  const matchNum = (re: RegExp) => {
    const matched = text.match(re)?.[1] || '0';
    const cleanNum = matched.toString().replace(/[^0-9.]/g, '');
    return Number(cleanNum) || 0; };
  const invoice_patterns = [
    /Invoice\s*(?:No\.|Number|#)?\s*:?\s*([A-Za-z0-9-]+)/i,
    /Bill\s*(?:No\.|Number|#)?\s*:?\s*([A-Za-z0-9-]+)/i,
    /Receipt\s*(?:No\.|Number|#)?\s*:?\s*([A-Za-z0-9-]+)/i,
    /(?:^|\n)(?:INV|REC|BILL)[\s-]*([A-Za-z0-9-]+)/i,
    /(?:^|\n)([0-9]{4,})/m];
  const vendor_patterns = [
    /^([A-Z][A-Za-z\s&.,]+ Limited)/m,
    /^([A-Z][A-Za-z\s&.,]+ Inc\.?)/m,
    /^([A-Z][A-Za-z\s&.,]+ Corp\.?)/m,
    /^([A-Z][A-Za-z\s&.,]+ LLC)/m,
    /^([A-Z][A-Za-z\s&.,]+ Ltd\.?)/m,
    /From[:\s]*([^\n\r]+)/i,
    /Vendor[:\s]*([^\n\r]+)/i,
```

```
Supplier[:\s]*([^\n\r]+)/i,
Company[:\s]*([^\n\r]+)/i,
([A-Z][A-Za-z\s&.,]+ (?:Limited|Ltd|Inc|Corp|LLC|LLP))/];
  const customer_patterns = [
    /CLIENT\s+([^\n]+)/i,
    /Bill\s*To[:\s]*([^\n\r]+)/i,
    /Sold\s*To[:\s]*([^\n\r]+)/i,
    /Customer[:\s]*([^\n\r]+)/i,
    /Client[:\s]*([^\n\r]+)/i,
    /(?:^|\n)CLIENT[:\s]*([^\n]+)/im];
  const date_patterns = [
    /DATE\s+([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Invoice\s*Date[:\s]*([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Date[:\s]*([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Invoice\s*Date[:\s]*([0-9]{1,2}[\/\-][0-9]{1,2}[\/\-][0-9]{2,4})/i,
    /Date[:\s]*([0-9]{1,2}[\/\-][0-9]{1,2}[\/\-][0-9]{2,4})/i,
    /([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/,
    /([0-9]{1,2}[\/\-][0-9]{1,2}[\/\-][0-9]{2,4})/];
  const due_date_patterns = [
    /DUE\s*DATE\s+([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Due\s*Date[:\s]*([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Payment\s*Due[:\s]*([A-Za-z]{3,4}\.?\s+[0-9]{1,2},?\s+[0-9]{4})/i,
    /Due\s*Date[:\s]*([0-9]{1,2}[\/\-][0-9]{1,2}[\/\-][0-9]{2,4})/i,
    /Payment\s*Due[:\s]*([0-9]{1,2}[\/\-][0-9]{1,2}[\/\-][0-9]{2,4})/i];
  const total_patterns = [
    /TOTAL\s+\$([0-9,]+\.?[0-9]*)/i,
```

```
  /Total[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Grand\s*Total[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Amount\s*Due[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Final\s*Amount[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Total[:\s]*([$₹€£]?\s*[0-9,]+\.?[0-9]*)/i,
  /(?:Total|TOTAL)\s+([0-9,]+\.?[0-9]*)/i];
const subtotal_patterns = [
  /Subtotal[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Sub\s*Total[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Net\s*Amount[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Subtotal[:\s]*([$₹€£]?\s*[0-9,]+\.?[0-9]*)/i];
const gst_patterns = [
  /GST[:\s]*([0-9]{1,2})%/i,
  /Tax[:\s]*([0-9]{1,2})%/i,
  /VAT[:\s]*([0-9]{1,2})%/i];
const gst_amount_patterns = [
  /GST(?:\s*Amount)?[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /Tax(?:\s*Amount)?[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /VAT(?:\s*Amount)?[:\s]*\$([0-9,]+\.?[0-9]*)/i,
  /GST(?:\s*Amount)?[:\s]*([$₹€£]?\s*[0-9,]+\.?[0-9]*)/i];
const tryPatterns = (patterns: RegExp[]) => {
  for (const pattern of patterns) {
    const result = match(pattern);
    if (result) return result;}
  return '';};
const tryNumPatterns = (patterns: RegExp[]) => {
  for (const pattern of patterns) {
```

```
      const result = matchNum(pattern);
      if (result > 0) return result;}
    return 0;};
  const emails: string[] = [];
  let emailMatch;
  const emailRegex = /([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})/g;
  while ((emailMatch = emailRegex.exec(text)) !== null && emails.length < 5)
{emails.push(emailMatch[1]);}
  const invoice_number = tryPatterns(invoice_patterns);
  const vendor_name = tryPatterns(vendor_patterns);
  const customer_name = tryPatterns(customer_patterns);
  const vendor_email = emails[0] || '';
  const customer_email = emails[1] || '';
  const invoice_date_raw = tryPatterns(date_patterns);
  const due_date_raw = tryPatterns(due_date_patterns);
  const formatDate = (dateStr: string) => {
    if (!dateStr) return new Date().toISOString().split('T')[0];
    try {
      if (dateStr.match(/[A-Za-z]/)) {
        const monthMap: { [key: string]: string } = {
          'jan': '01', 'january': '01',
          'feb': '02', 'february': '02',
          'mar': '03', 'march': '03',
          'apr': '04', 'april': '04',
          'may': '05',
          'jun': '06', 'june': '06',
          'jul': '07', 'july': '07',
```

```
        'aug': '08', 'august': '08',
        'sep': '09', 'sept': '09', 'september': '09',
        'oct': '10', 'october': '10',
        'nov': '11', 'november': '11',
        'dec': '12', 'december': '12'};
      const m = dateStr.match(/([A-Za-z]+)\.?\s+([0-9]{1,2}),?\s+([0-9]{4})/);
      if (m) {
        const [, monthStr, day, year] = m;
        const month = monthMap[monthStr.toLowerCase().replace('.', '')];
        if (month) return `${year}-${month}-${day.padStart(2, '0')}`; }}
    const cleanDate = dateStr.replace(/[\s\-]/g, '/');
    const parsed = new Date(cleanDate);
    if (isNaN(parsed.getTime())) return new Date().toISOString().split('T')[0];
    return parsed.toISOString().split('T')[0];
  } catch {
    return new Date().toISOString().split('T')[0]; }};
  const invoice_date = formatDate(invoice_date_raw);
  const due_date = formatDate(due_date_raw) || new Date(Date.now() + 30 *
86400000).toISOString().split('T')[0];
  const subtotal = tryNumPatterns(subtotal_patterns);
  const gst_percentage = tryNumPatterns(gst_patterns);
  const gst_amount = tryNumPatterns(gst_amount_patterns);
  const total_amount = tryNumPatterns(total_patterns) || (subtotal +
gst_amount);
  const items: OCRItem[] = [];
  const tablePattern = /([0-9]{3,})\s+([A-Za-z0-9\s&\-\.]+?)\s+\$([0-9,]+(?:\.[0-
9]{2})?)\s+([0-9]+)\s+\$([0-9,]+(?:\.[0-9]{2})?)/g;
```

```
  let itemMatch: RegExpExecArray | null;
  while ((itemMatch = tablePattern.exec(text)) !== null && items.length < 20) {
    const [, itemCode, description, rate, quantity, amount] = itemMatch;
    items.push({
      name: `${itemCode} ${description.trim()}`,
      quantity: parseInt(quantity) || 1,
      rate: parseFloat(rate.replace(/,/g, '')) || 0,
      amount: parseFloat(amount.replace(/,/g, '')) || 0});}
  if (items.length === 0) {
    const simplePattern = /([A-Za-z][A-Za-z\s]{5,40})\s+([0-9]+)\s+\$?([0-
9.,]+)\s+\$?([0-9.,]+)/g;
    while ((itemMatch = simplePattern.exec(text)) !== null && items.length10)
{const [, name, qty, rate, amount] = itemMatch;
      items.push({
        name: name.trim(),
        quantity: parseInt(qty) || 1,
        rate: parseFloat(rate.replace(/,/g, '')) || 0,
        amount: parseFloat(amount.replace(/,/g, '')) || 0});}}
  return {
    invoice_number,
    total_amount,
    invoice_date,
    due_date,
    vendor_name,
    vendor_email,
    customer_name,
    customer_email,
```

```
    subtotal,
    gst_percentage,
    gst_amount, items}; }
```

CRM Process:

```
interface Customer {
  id: string;
  company_name: string;
  contact_name?: string;
  email?: string;
  phone?: string;
  address?: string;
  gst_number?: string;
  created_at: string;}
const Customers = () => {
  const [customers, setCustomers] = useState<Customer[]>([]);
  const [loading, setLoading] = useState(true);
  const [searchTerm, setSearchTerm] = useState("");
  const [showAddDialog, setShowAddDialog] = useState(false);
  const { toast } = useToast();
  useEffect(() => {
    fetchCustomers();}, []);
  const fetchCustomers = async () => {
    try {
      const { data, error } = await supabase
        .from('customers')
        .select('*')
```

```
        .order('created_at', { ascending: false });
      if (error) throw error;
      setCustomers(data || []);
    } catch (error) {
      console.error('Error fetching customers:', error);
      toast({
        title: "Error",
        description: "Failed to load customers",
        variant: "destructive",});
    } finally {
      setLoading(false);}};
  const handleCustomerUpdate = (updatedCustomer: Customer) => {
    setCustomers((prev) => prev.map((customer) => customer.id ===
updatedCustomer.id ? updatedCustomer : customer));
    toast({ title: "Customer updated successfully" });};
  const handleCustomerDelete = (customerId: string) => {
    setCustomers((prev) => prev.filter((customer) => customer.id !==
customerId)); };
  const filteredCustomers = customers.filter(customer =>

customer.company_name.toLowerCase().includes(searchTerm.toLowerCase())
||

customer.contact_name?.toLowerCase().includes(searchTerm.toLowerCase()) ||
    customer.email?.toLowerCase().includes(searchTerm.toLowerCase()));
  return (
    <div className="space-y-6">
```

```
      {/* Header */}
      <div className="flex flex-col sm:flex-row justify-between items-start
sm:items-center gap-4">
        <div>
          <h1 className="text-3xl font-bold text-foreground">Customer
Management</h1>
          <p className="text-muted-foreground mt-1">
            Manage your customer database and CRM records
          </p>
        </div>
        <Button
          className="shadow-corporate"
          onClick={() => setShowAddDialog(true)}>
          <Plus className="w-4 h-4 mr-2" />
          Add Customer
        </Button>
      </div>
      {/* Search */}
      <Card className="shadow-card border-border">
        <CardContent className="p-4">
          <div className="relative">
            <Search className="absolute left-3 top-1/2 transform -translate-y-1/2
text-muted-foreground w-4 h-4" />
            <Input
              placeholder="Search customers by name, contact, or email..."
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
```

```
            className="pl-10"/>
          </div>
        </CardContent>
      </Card>
      {/* Customers Table */}
      <Card className="shadow-card border-border">
        <CardHeader>
          <CardTitle className="flex items-center gap-2">
            <Building2 className="w-5 h-5 text-corporate-blue" />
            Customers ({filteredCustomers.length})
          </CardTitle>
        </CardHeader>
        <CardContent>
          {loading ? (
            <div className="flex items-center justify-center h-64">
              <div className="text-center">
                <div className="animate-spin rounded-full h-8 w-8 border-b-2
border-corporate-blue mx-auto"></div>
                <p className="text-muted-foreground mt-2">Loading
customers...</p>
              </div>
            </div>
          ) : filteredCustomers.length === 0 ? (
            <div className="text-center py-12">
              <Building2 className="w-12 h-12 text-muted-foreground mx-auto
mb-4" />
              <h3 className="font-medium text-foreground">No customers
```

```
found</h3>
            <p className="text-muted-foreground">
              {searchTerm ? "Try adjusting your search" : "Add your first customer
to get started"}</p> </div>) : (
          <div className="rounded-lg border border-border overflow-hidden">
            <Table>
              <TableHeader>
                <TableRow className="bg-muted/50">
                  <TableHead className="font-medium">Company</TableHead>
                  <TableHead className="font-medium">Contact
Person</TableHead>
                  <TableHead className="font-medium">Email</TableHead>
                  <TableHead className="font-medium">Phone</TableHead>
                  <TableHead className="font-medium">GST
Number</TableHead>
                  <TableHead className="font-medium">Added</TableHead>
                  <TableHead className="font-medium">Actions</TableHead>
                </TableRow>
              </TableHeader>
              <TableBody>
                {filteredCustomers.map((customer) => (
                  <TableRow key={customer.id} className="hover:bg-muted/30">
                    <TableCell>
                      <div className="flex items-center gap-3">
                        <div className="w-8 h-8 rounded-lg bg-corporate-light-
blue/20 flex items-center justify-center">
                          <Building2 className="w-4 h-4 text-corporate-blue" />
```

```
                    </div>
                    <div>
                      <p className="font-medium text-
foreground">{customer.company_name}</p>
                      {customer.address && (
                        <p className="text-xs text-muted-foreground truncate max-
w-xs">
                          {customer.address}
                        </p> )}
                    </div>
                  </div>
                </TableCell>
                <TableCell>
                  {customer.contact_name ? (
                    <div className="flex items-center gap-2">
                      <div className="w-6 h-6 rounded-full bg-muted flex items-
center justify-center">
                        <span className="text-xs font-medium">
                          {customer.contact_name.charAt(0).toUpperCase()}
                        </span>
                      </div>
                      <span>{customer.contact_name}</span>
                    </div>) : (
                    <span className="text-muted-foreground">N/A</span>}]
                </TableCell>
                <TableCell>
                  {customer.email ? (
```

```
    <div className="flex items-center gap-2">
      <Mail className="w-4 h-4 text-muted-foreground" />
      <span className="text-sm">{customer.email}</span>
    </div>) : (
    <span className="text-muted-foreground">N/A</span>)}
</TableCell>
<TableCell>
  {customer.phone ? (
    <div className="flex items-center gap-2">
      <Phone className="w-4 h-4 text-muted-foreground" />
      <span className="text-sm">{customer.phone}</span>
    </div>) : (
    <span className="text-muted-foreground">N/A</span>)}
</TableCell>
<TableCell>
  {customer.gst_number ? (
    <Badge variant="outline" className="font-mono text-xs">
      {customer.gst_number}
    </Badge>) : (
    <span className="text-muted-foreground">N/A</span>)}
</TableCell>
<TableCell className="text-muted-foreground">
  {new Date(customer.created_at).toLocaleDateString()}
</TableCell>
<TableCell>
  <CustomerActionsDropdown
    customer={customer}
```

```
                onUpdate={handleCustomerUpdate}
                onDelete={handleCustomerDelete} />
              </TableCell>
            </TableRow>}}]
          </TableBody>
        </Table>
      </div>)}
    </CardContent>
  </Card>
  <AddCustomerDialog
    open={showAddDialog}
    onOpenChange={setShowAddDialog}
    onCustomerAdded={fetchCustomers}/>
</div>);};
export default Customers;
```

Analysis Dashboard:

```
const Analytics = () => {
  return (
    <div className="space-y-6">
      {/* Header */}
      <div>
        <h1 className="text-3xl font-bold text-foreground">Analytics &
Reports</h1>
        <p className="text-muted-foreground mt-1">
          Comprehensive insights into your invoice processing and CRM data
        </p>
```

```
{/* Overview Stats */}
<DashboardStats />
{/* Charts Grid */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
  {/* Revenue Trends */}
  <Card className="shadow-card border-border">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <TrendingUp className="w-5 h-5 text-corporate-blue" />
        Revenue Trends (Monthly)
      </CardTitle>
    </CardHeader>
    <CardContent>
      <RevenueChart />
    </CardContent>
  </Card>
  {/* Invoice Status Distribution */}
  <Card className="shadow-card border-border">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <PieChart className="w-5 h-5 text-success" />
        Invoice Status Distribution
      </CardTitle>
    </CardHeader>
    <CardContent>
      <StatusPieChart />
    </CardContent></Card>
```

```
  {/* GST Collection Summary */}
  <Card className="shadow-card border-border">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <BarChart3 className="w-5 h-5 text-warning" />
        GST Collection Summary
      </CardTitle>
    </CardHeader>
    <CardContent>
      <GSTBarChart />
    </CardContent> </Card>
  {/* Processing Timeline */}
  <Card className="shadow-card border-border">
    <CardHeader>
      <CardTitle className="flex items-center gap-2">
        <Calendar className="w-5 h-5 text-info" />
        Processing Timeline
      </CardTitle>
    </CardHeader>
    <CardContent>
      <ProcessingTimelineChart />
    </CardContent></Card></div>
{/* Summary Reports */}
<div className="grid grid-cols-1 md:grid-cols-3 gap-6">
  <TopCustomersCard />
  <TopVendorsCard />
  <RecentActivityCard />
```

```
    </div></div>);};
// Chart Components
function RevenueChart() {
  const [data, setData] = useState<Array<{ month: string; revenue: number
}>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const fetchData = async () => {
      try {
        const { data: invoices, error } = await supabase
          .from('invoices')
          .select('total_amount, created_at')
          .eq('status', 'paid');
        if (error) throw error;
        // Group by month
        const monthlyData: Record<string, number> = {};
        (invoices || []).forEach((invoice: any) => {
          const month = new Date(invoice.created_at).toLocaleDateString('en-US',
{ year: 'numeric', month: 'short' });
          monthlyData[month] = (monthlyData[month] || 0) +
Number(invoice.total_amount);});
        const chartData = Object.entries(monthlyData)
          .map(([month, revenue]) => ({ month, revenue }))
          .sort((a, b) => new Date(a.month).getTime() - new
Date(b.month).getTime())
          .slice(-6); // Last 6 months
```

```
      setData(chartData);
    } finally {
      setLoading(false);}};
  fetchData();
 }, []);
 if (loading) return <div className="h-80 flex items-center justify-
center">Loading...</div>;
 return (
  <div className="h-80">
    <ResponsiveContainer width="100%" height="100%">
      <LineChart data={data}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="month" />
        <YAxis />
        <Tooltip formatter={(value) => [`$${value}`, 'Revenue']} />
        <Line type="monotone" dataKey="revenue" stroke="hsl(var(--corporate-
blue))" strokeWidth={2} />
      </LineChart>
    </ResponsiveContainer></div>);}
function StatusPieChart() {
 const [data, setData] = useState<Array<{ name: string; value: number; color:
string }>>([]);
 const [loading, setLoading] = useState(true);
 useEffect(() => {
  const fetchData = async () => {
    try {
      const { data: invoices, error } = await supabase
```

```
        .from('invoices')
        .select('status');
      if (error) throw error;
      const statusCounts: Record<string, number> = {};
      (invoices || []).forEach((invoice: any) => {
        statusCounts[invoice.status] = (statusCounts[invoice.status] || 0) + 1;});
      const colors = {
        paid: 'hsl(var(--success))',
        pending: 'hsl(var(--warning))',
        overdue: 'hsl(var(--destructive))',
        validated: 'hsl(var(--info))',
        processed: 'hsl(var(--corporate-blue))'};
      const chartData = Object.entries(statusCounts).map(([status, count]) => ({
        name: status.charAt(0).toUpperCase() + status.slice(1),
        value: count,
        color: colors[status as keyof typeof colors] || 'hsl(var(--muted))'}));
      setData(chartData);} finally {
      setLoading(false);}};
    fetchData();}, []);
  if (loading) return <div className="h-80 flex items-center justify-
center">Loading...</div>;
  return (
    <div className="h-80">
      <ResponsiveContainer width="100%" height="100%">
        <RechartsPieChart>
          <Pie
            data={data}
```

```
          cx="50%"
          cy="50%"
          outerRadius={80}
          fill="#8884d8"
          dataKey="value"
          label={(entry) => `${entry.name}: ${entry.value}`}>
          {data.map((entry, index) => (
            <Cell key={`cell-${index}`} fill={entry.color} />))}
        </Pie>
        <Tooltip />
      </RechartsPieChart>
    </ResponsiveContainer>
  </div>);}
function GSTBarChart() {
  const [data, setData] = useState<Array<{ month: string; gst: number }>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const fetchData = async () => {
      try {
        const { data: invoices, error } = await supabase
          .from('invoices')
          .select('gst_amount, created_at')
          .not('gst_amount', 'is', null);
        if (error) throw error;
        const monthlyGST: Record<string, number> = {};
        (invoices || []).forEach((invoice: any) => {
          const month = new Date(invoice.created_at).toLocaleDateString('en-US',
```

```
{ year: 'numeric', month: 'short' });
        monthlyGST[month] = (monthlyGST[month] || 0) +
Number(invoice.gst_amount || 0);});
      const chartData = Object.entries(monthlyGST)
        .map(([month, gst]) => ({ month, gst }))
        .sort((a, b) => new Date(a.month).getTime() - new
Date(b.month).getTime())
        .slice(-6);
      setData(chartData);
    } finally {
      setLoading(false);}};
  fetchData();}, []);
 if (loading) return <div className="h-80 flex items-center justify-
center">Loading...</div>;
 return (
  <div className="h-80">
    <ResponsiveContainer width="100%" height="100%">
      <BarChart data={data}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="month" />
        <YAxis />
        <Tooltip formatter={(value) => [`$${value}`, 'GST Collected']} />
        <Bar dataKey="gst" fill="hsl(var(--warning))" />
      </BarChart>
    </ResponsiveContainer>
  </div>);}
```

```
function ProcessingTimelineChart() {
  const [data, setData] = useState<Array<{ date: string; processed: number
}>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const fetchData = async () => {
      try {
        const { data: invoices, error } = await supabase
          .from('invoices')
          .select('created_at');
        if (error) throw error;
        const dailyCounts: Record<string, number> = {};
        (invoices || []).forEach((invoice: any) => {
          const date = new Date(invoice.created_at).toLocaleDateString();
          dailyCounts[date] = (dailyCounts[date] || 0) + 1;});
        const chartData = Object.entries(dailyCounts)
          .map(([date, processed]) => ({ date, processed }))
          .sort((a, b) => new Date(a.date).getTime() - new Date(b.date).getTime())
          .slice(-14); // Last 14 days
        setData(chartData);
      } finally {
        setLoading(false);}};
    fetchData();}, []);
  if (loading) return <div className="h-80 flex items-center justify-
center">Loading...</div>;
  return (
    <div className="h-80">
```

```
      <ResponsiveContainer width="100%" height="100%">
        <LineChart data={data}>
          <CartesianGrid strokeDasharray="3 3" />
          <XAxis dataKey="date" />
          <YAxis />
          <Tooltip formatter={(value) => [`${value}`, 'Invoices Processed']} />
          <Line type="monotone" dataKey="processed" stroke="hsl(var(--info))"
strokeWidth={2} />
        </LineChart>
      </ResponsiveContainer>
    </div>);}
export default Analytics;
function currency(amount: number) {
  return new Intl.NumberFormat(undefined, { style: "currency", currency:
"USD" }).format(amount);}
function TopCustomersCard() {
  const [rows, setRows] = useState<Array<{ name: string; total: number; count:
number }>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const load = async () => {
      try {
        const { data, error } = await supabase
          .from("invoices")
          .select("total_amount, customers:customer_id(company_name)")
          .eq("status", "paid");
        if (error) throw error;
```

```
      const aggregates: Record<string, { total: number; count: number }> = {};
      (data || []).forEach((row: any) => {
        const name = row.customers?.company_name || "Unknown";
        if (!aggregates[name]) aggregates[name] = { total: 0, count: 0 };
        aggregates[name].total += Number(row.total_amount || 0);
        aggregates[name].count += 1;});
      const top = Object.entries(aggregates)
        .map(([name, v]) => ({ name, total: v.total, count: v.count }))
        .sort((a, b) => b.total - a.total)
        .slice(0, 5);
      setRows(top);
    } finally {
      setLoading(false);}};
  load();}, []);
 return (
  <Card className="shadow-card border-border">
    <CardHeader className="pb-3">
      <CardTitle className="text-lg">Top Customers</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {loading && <div className="text-sm text-muted-
foreground">Loading...</div>}
      {!loading && rows.length === 0 && <div className="text-sm text-
muted-foreground">No data</div>}
      {rows.map((row, index) => (
        <div key={index} className="flex items-center justify-between p-3
rounded-lg bg-muted/30">
```

```
          <div>
            <p className="font-medium text-foreground">{row.name}</p>
            <p className="text-sm text-muted-foreground">{row.count}
invoice(s)</p>
          </div>
          <div className="text-right">
            <p className="font-semibold text-
foreground">{currency(row.total)}</p>
          </div>
        </div>))}
      </CardContent>
    </Card>);}
function TopVendorsCard() {
  const [rows, setRows] = useState<Array<{ name: string; total: number; count:
number }>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const load = async () => {
      try {
        const { data, error } = await supabase
          .from("invoices")
          .select("total_amount, vendors:vendor_id(company_name)")
          .eq("status", "paid");
        if (error) throw error;
        const aggregates: Record<string, { total: number; count: number }> = {};
        (data || []).forEach((row: any) => {
          const name = row.vendors?.company_name || "Unknown";
```

```
        if (!aggregates[name]) aggregates[name] = { total: 0, count: 0 };
        aggregates[name].total += Number(row.total_amount || 0);
        aggregates[name].count += 1;});
      const top = Object.entries(aggregates)
        .map(([name, v]) => ({ name, total: v.total, count: v.count }))
        .sort((a, b) => b.total - a.total)
        .slice(0, 5);
      setRows(top);
    } finally {
      setLoading(false);}};
  load();}, []);
 return (
  <Card className="shadow-card border-border">
    <CardHeader className="pb-3">
      <CardTitle className="text-lg">Top Vendors</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {loading && <div className="text-sm text-muted-
foreground">Loading...</div>}
      {!loading && rows.length === 0 && <div className="text-sm text-
muted-foreground">No data</div>}
      {rows.map((row, index) => (
        <div key={index} className="flex items-center justify-between p-3
rounded-lg bg-muted/30">
          <div>
            <p className="font-medium text-foreground">{row.name}</p>
            <p className="text-sm text-muted-foreground">{row.count}
```

```
invoice(s)</p>
          </div>
          <div className="text-right">
            <p className="font-semibold text-
foreground">{currency(row.total)}</p>
          </div>
        </div>))}
      </CardContent>
    </Card>);}
function RecentActivityCard() {
  const [rows, setRows] = useState<Array<{ action: string; time: string; type:
"success" | "info" }>>([]);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const load = async () => {
      try {
        const { data, error } = await supabase
          .from("invoices")
          .select("invoice_number,status,updated_at")
          .order("updated_at", { ascending: false })
          .limit(6);
        if (error) throw error;
        const mapped = (data || []).map((row) => {
          const type: "success" | "info" = row.status === "paid" ? "success" :
"info";
          const action = row.status === "paid"
            ? `Invoice ${row.invoice_number} paid`
```

```
          : row.status === "validated"
            ? `Invoice ${row.invoice_number} validated`
            : row.status === "processed"
              ? `Invoice ${row.invoice_number} processed`
              : `Invoice ${row.invoice_number} created`;
        const time = new Date(row.updated_at as unknown as
string).toLocaleString();
        return { action, time, type };});
      setRows(mapped);
    } finally {
      setLoading(false);}};
  load();}, []);
 return (
  <Card className="shadow-card border-border">
    <CardHeader className="pb-3">
      <CardTitle className="text-lg">Recent Activity</CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
      {loading && <div className="text-sm text-muted-
foreground">Loading...</div>}
      {!loading && rows.length === 0 && <div className="text-sm text-
muted-foreground">No activity</div>}
      {rows.map((activity, index) => (
        <div key={index} className="flex items-start gap-3 p-3 rounded-lg bg-
muted/30">
          <div className={`w-2 h-2 rounded-full mt-2 ${activity.type ===
"success" ? "bg-success" : "bg-info"}`} />
```

REFERENCES:

1. Kumar Subham, "Integrating AI into CRM Systems for Enhanced Customer Retention", Journal of Information Systems Engineering & Management 10(45s): 474-489, DOI: 10.52783/jisem.v10i45s.8892, April 2025.

2. Aziz Amari, Mariem Makni, Wissal Fnaich, Akram Lahmar, Fedi Koubaa, Oumayma Charrad, Mohamed Ali Zormati, Rabaa Youssef, "An Efficient Deep Learning-Based Approach to Automating Invoice Document Validation", National Institute of Applied Sciences and Technology(INSAT) University of Carthage Tunis, Tunisia, DOI:10.48550/arXiv.2503.12267, March 2025.

3. Jayesh Jhurani, "Enhancing Customer Relationship Management in ERP Systems through AI: Personalized Interactions, Predictive Modeling, and Service Automation", International Journal of Science and Research (IJSR) 13(3): 892-897, DOI: 10.21275/SR24313021820, March 2024.

4. Srinivasa Rao Gunturu, Madhavi Godbole, Hari Prasad Josyula, "FinTech-Automatic Payment Process in the ERP System", International Journal of Computer Trends and Technology 72(1):99-103, DOI: 10.14445/22312803/IJCTT-V72I11P116, February 2024.

5. Adem Akdogan, Murat Kurt, "ExTTNet: A Deep Learning Algorithm for Extracting Table Texts from Invoice Images", arXiv:2402.02246, https://doi.org/10.48550/arXiv.2402.02246, February 2024.

6. Rashi, Dr Biplab, Kumar Biswal, Srinivasa Rao Yadavalli, "An AI-Based Customer Relationship Management Framework for Business Applications", International Journal of Intelligent Systems and Applications in Engineering 12(12): 686-695, January 2024.

7. Thomas Saout, Frederic Lardeux, Frederic Saubion, “An Overview of Data Extraction from Invoices”, IEEE Access, PP(99):1-1 DOI: 10.1109/ACCESS.2024.3360528. January 2024.

8. Jerome Blanchard, Yolande Belaid, Abdel Belaid, “Automatic Generation of a Custom Corpora for Invoice Analysis and Recognition”, 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW), DOI: 10.1109/ICDARW.2019.60121, September 2019.

9. D. Lohani, A. Bela¨ıd and Y. Bela¨ıd. “An Invoice Reading System using a Graph Convolutional Network”. IWRR, Dec 2018, PERTH, Australia. 2018.

10. B. Klein, S. Agne and A. Dengel: “Results of a study on invoicereading systems in Germany”. DAS, pp. 451-462, 2004.

11. D. Nadeau and S. Sekine: “A survey of named entity recognition and classification”. Lingvisticae Investigationes, 30, pp. 3-26, 2007.

12. D. Schuster and K. Muthmann and D. Esser and A. Schill and M. Berger and C. Weidling and K. Aliyev and A. Hofmeier: “Intellix-end-user trained information extraction for document archiving”. ICDAR, 2013, pp. 101-105.

13. M. Rusinol, T. Benkhelfallah and V. Poulain d’Andecy: “Field extraction from administrative documents by incremental structural templates”. ICDAR, 2013, pp. 1100-1104.

14. H. Hamza, Y. Bela¨ıd and A. Bela¨ıd: “A case-based reasoning approach for invoice structure extraction”. ICDAR, 2007, pp.327-331.

15. A. Dengel and B. Klein: smartfix: “A requirements-driven system for document analysis and understanding”. DAS, 2002, pp. 433-444.

16. F. Cesarini, E. Francesconi, M. Gori and G. Soda: “Analysis and understanding of multi-class invoices”. ICDAR, 2003, pp. 102-114.

17. V. P. d’Andecy, E. Hartman and M. Rusinol: “Field extraction by hybrid incremental and a priori structural templates”. DAS, 2018, pp. 251-256.

18. R. B. Palm, O. Winther and F. Laws: “Cloudscan a configuration-free invoice analysis system using recurrent neural networks”. ICDAR 2017, pp. 406-413.

BOOK REFERENCES:

1. Python: "Python Crash Course" by Eric Matthes.
2. Flask: "Flask Web Development: Developing Web Applications with Python" by Miguel Grinberg.
3. MySQL: "Learning MySQL" by Robin Nixon.
4. WampServer: "WampServer: Windows Apache MySQL PHP" by David Sayre.
5. TensorFlow: "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron.
6. Pandas: "Python for Data Analysis" by Wes McKinney.
7. Scikit-Learn: "Introduction to Machine Learning with Python: A Guide for Data Scientists" by Andreas C. Müller and Sarah Guido.
8. Matplotlib: "Python Data Visualization Cookbook" by Igor Milovanović.
9. NumPy: "Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib" by Robert Johansson.
10. OpenCV: "OpenCV 4 for Secret Agents" by Joseph Hows

WEB REFERENCES

1. Python Documentation: https://docs.python.org/
2. Flask Documentation: https://flask.palletsprojects.com/en/2.0.x/
3. MySQL Documentation: https://dev.mysql.com/doc/
4. TensorFlow Documentation: https://www.tensorflow.org/guide
5. Pandas Documentation: https://pandas.pydata.org/docs/
6. Scikit-Learn Documentation: https://scikitlearn.org/stable/documentation.html
7. Matplotlib Documentation: https://matplotlib.org/stable/contents.html
8. NumPy Documentation: https://numpy.org/doc/